

МЕТОДИЧЕСКОЕ ПОСОБИЕ

PostgreSQL с нуля

(теория + практика в Telegram-боте SQLearningBot)

Данное методическое пособие поможет Вам освоить язык SQL на примере PostgreSQL. Теория подкрепляется практикой в Telegram-боте SQLearningBot, который проверяет запросы и даёт подсказки.

Автор-составитель:

Студент 3 курса
Направления 09.03.02 информационные системы и технологии
Профиля «информационные технологии в образовании»

Ваганов Владислав Асымович

На основе работы:

«Обучение проектированию и разработке
баз данных в MySQL в рамках курса
по выбору для технологического профиля»

Касимовской О.А.

Оглавление

Оглавление	2
Введение	3
Раздел 1. Основы реляционных баз данных и языка SQL	5
1.1. Что такое база данных и СУБД	5
1.2. Реляционная модель данных	5
1.3. Ключи и связи между таблицами	6
1.4. Язык SQL	7
1.5. Типы данных в PostgreSQL	8
1.6. Нормализация баз данных	11
1.7. Учебная база данных «Library»	11
Раздел 2. Тематические занятия по SQL в PostgreSQL	13
Занятие 1. Создание и удаление таблиц в PostgreSQL	13
Занятие 2. Вставка, обновление и удаление данных (INSERT, UPDATE, DELETE)	18
Занятие 3. Выборка данных: команда SELECT	22
Занятие 4. Сортировка и ограничение количества записей (ORDER BY, LIMIT)	26
Занятие 5. Агрегатные функции и группировка (COUNT, SUM, AVG, GROUP BY, HAVING)	30
Занятие 6. Объединение таблиц (JOIN)	34
Занятие 7. Подзапросы (вложенные запросы)	38
Занятие 8. Представления (VIEW)	42
Занятие 9. Индексы и оптимизация запросов	47
Занятие 10. Транзакции (BEGIN, COMMIT, ROLLBACK)	52
Занятие 11. Итоговый проект: «Библиотечная аналитика»	58
Заключение	62
Приложение А (справочное)	63
А.1. Схема базы данных	63
А.2. Полный SQL-скрипт для создания БД	64
А.3. Начальное наполнение данными	65
Приложение Б. Часто задаваемые вопросы (FAQ)	67
Приложение В. Глоссарий основных терминов	69
Список использованных источников	71

Введение

Данная методичка является дополнительным материалом к выпускной квалификационной работе Вагапова В.А. на тему «Разработка чат-бота на основе мессенджера Telegram для освоения языка запросов». Основная цель работы – создание Telegram-бота SQLearningBot, который помогает пользователям изучать структурированный язык запросов SQL в интерактивном режиме с автоматической проверкой решений через внешний AI API.

Методичка не дублирует функционал бота, а **дополняет его**. Если бот отвечает за практику (решение задач, проверку кода, подсказки, статистику), то методичка даёт систематизированную теоретическую базу. В ней компактно и последовательно изложены основные понятия реляционных баз данных, правила проектирования и, главное, синтаксис и примеры SQL-запросов в **PostgreSQL** – именно эта СУБД используется как в самом боте (при хранении данных о пользователях и прогрессе), так и рекомендуется для самостоятельного изучения.

При разработке методички за основу взята работа Касимовской О.А. «Обучение проектированию и разработке баз данных в MySQL в рамках курса по выбору для технологического профиля» (МПГУ, 2021). Однако исходный материал **полностью адаптирован под PostgreSQL**:

- все примеры синтаксиса переписаны на диалект PostgreSQL;
- учтены особенности PostgreSQL (тип SERIAL, ограничения, работа с транзакциями, триггеры на PL/pgSQL и т.д.);
- сохранена структура «общая теория → тематические занятия», но практические задания вынесены в бот.

Таким образом, методичка и бот образуют единый учебный контур:

1. Пользователь читает тему в методичке, знакомится с теорией и примерами.
2. Затем открывает Telegram-бота и решает задачи, привязанные к этой теме (номера задач указаны в конце каждой темы).
3. В случае ошибок бот даёт развёрнутую обратную связь (через AI-модель) и подсказки.

Такой подход сочетает преимущества традиционного учебного текста (структурированность, возможность перечитывать, отсутствие спешки) и интерактивного тренажёра (мгновенная проверка, диалоговый режим, геймификация).

Методичка построена следующим образом.

- **Раздел 1. Основы реляционных баз данных и языка SQL.** Здесь вводятся ключевые понятия: база данных, СУБД, реляционная модель,

таблицы, ключи, связи, типы данных, нормализация. Материал даётся в объёме, достаточном для понимания последующих тем.

- **Раздел 2. Тематические занятия по SQL в PostgreSQL.** Всего предусмотрено **11 занятий**:
 - **Занятия 1–10** посвящены основным группам операторов: создание и удаление таблиц, манипулирование данными, выборка, фильтрация, сортировка, агрегатные функции, объединения (JOIN), подзапросы, представления (VIEW), индексы и оптимизация, транзакции.
 - **Занятие 11 – итоговый проект**, закрепляющий все изученные конструкции. Оно не вводит новой теории, а предлагает решить комплексную задачу на основе учебной базы данных «Library», используя все изученные конструкции. Это позволяет закрепить навыки и получить законченный результат.
В конце каждого занятия приведена ссылка на соответствующие номера задач в боте SQLearningBot.
- **Заключение.** Подводятся итоги пройденного курса, даются рекомендации по дальнейшему самостоятельному изучению SQL.
- **Приложения.** Включают полное описание учебной базы данных «Library» (PostgreSQL-версия), SQL-скрипты для её создания, ответы на частые вопросы (FAQ), краткий глоссарий и список рекомендуемых ресурсов.

Все теоретические положения, если они прямо заимствованы из исходной работы Касимовской О.А. или из официальной документации PostgreSQL, сопровождаются ссылками. Список литературы приведён в конце методички.

Методичка предназначена для **любых пользователей**, желающих освоить SQL с нуля или систематизировать уже имеющиеся знания. Для работы достаточно иметь доступ к Telegram-боту @SQLearningBot (или его локальной версии). Устанавливать и настраивать PostgreSQL на своём компьютере не требуется – все проверочные запросы анализируются ботом, а для экспериментов можно использовать встроенный в бота режим или бесплатные онлайн-песочницы (рекомендации даны в приложении).

Раздел 1. Основы реляционных баз данных и языка SQL

Прежде чем переходить к практическим занятиям и решению задач в боте, необходимо усвоить базовые понятия. Этот раздел даёт тот минимум теории, без которого дальнейшая работа с SQL будет затруднительна. Если какие-то термины покажутся сложными, вы всегда можете вернуться к ним по мере прохождения занятий.

1.1. Что такое база данных и СУБД

База данных (БД) – это организованная структура для хранения информации. Простыми словами, это набор данных, который хранится в упорядоченном виде и позволяет быстро находить нужные сведения. Примерами баз данных в повседневной жизни могут служить телефонная книга, школьный журнал, каталог книг в библиотеке.

Однако просто иметь данные недостаточно – нужно уметь ими управлять: добавлять новые, изменять, удалять, искать. Для этого существуют **системы управления базами данных (СУБД)**. СУБД – это программа (или набор программ), которая предоставляет удобный интерфейс для работы с БД.

В данном пособии мы будем использовать **PostgreSQL** – одну из самых мощных и популярных свободных реляционных СУБД. Она используется в крупных компаниях (Apple, Spotify, Uber) и государственных системах, а её документация считается эталонной.

1.2. Реляционная модель данных

Существует несколько моделей организации данных (иерархическая, сетевая, объектно-ориентированная), но самой распространённой является **реляционная модель**.

Термин «реляционная» происходит от английского *relation* – отношение. В реляционной модели все данные представляются в виде **таблиц** (отношений). Каждая таблица состоит из строк и столбцов.

- **Строка** (также называется *запись* или *кортеж*) – это один экземпляр объекта (одна книга, один автор, один читатель).
- **Столбец** (также называется *поле* или *атрибут*) – это характеристика объекта (название книги, год издания, фамилия автора).

На рисунке ниже показана структура реляционной таблицы (в текстовом виде):

Таблица "books" (книги)

id_book	name_book	id_author	year
1	Dubrovsky	1	2017
2	Eugene Onegin	1	2018
3	War and Peace	2	2015

Здесь:

- id_book – уникальный номер книги (первичный ключ),
- name_book – название,
- id_author – ссылка на автора,
- year – год издания.

1.3. Ключи и связи между таблицами

В реляционных базах данных таблицы редко существуют изолированно. Обычно они связаны друг с другом, чтобы избежать дублирования информации и обеспечить целостность.

1.3.1. Первичный ключ (Primary Key)

Первичный ключ – это столбец (или несколько столбцов), который однозначно идентифицирует каждую строку в таблице. Значения первичного ключа должны быть уникальными и не могут быть пустыми (NULL).

В примере выше id_book – первичный ключ таблицы books.

В PostgreSQL для автоматической генерации уникальных числовых ключей используется тип SERIAL (или BIGSERIAL для больших чисел). Например:

```
CREATE TABLE books (
  id_book SERIAL PRIMARY KEY,
  name_book VARCHAR(255) NOT NULL,
  year INT
);
```

1.3.2. Внешний ключ (Foreign Key)

Внешний ключ – это столбец (или набор столбцов), который ссылается на первичный ключ другой таблицы. Он устанавливает связь между таблицами.

В таблице books столбец id_author – внешний ключ, ссылающийся на id_author в таблице authors. Это означает, что каждая книга принадлежит какому-то автору, и нельзя добавить книгу с несуществующим id_author.

```
CREATE TABLE books (  
  id_book SERIAL PRIMARY KEY,  
  name_book VARCHAR(255),  
  id_author INT REFERENCES authors(id_author),  
  year INT  
);
```

1.3.3. Типы связей

Между таблицами могут быть три типа связей:

- **Один-к-одному (1:1)** – одной записи в первой таблице соответствует ровно одна запись во второй. Встречается редко (например, расширение основной таблицы).
- **Один-ко-многим (1:N)** – одной записи в первой таблице соответствует много записей во второй. Самый распространённый тип. Пример: один автор написал много книг.
- **Многие-ко-многим (M:N)** – одной записи в первой таблице соответствует много записей во второй, и наоборот. Реализуется через промежуточную таблицу. Пример: книга может быть выдана многим читателям, и читатель может взять много книг.

1.4. Язык SQL

SQL (Structured Query Language) – это структурированный язык запросов, предназначенный для работы с реляционными базами данных. Он является стандартом де-факто и поддерживается всеми популярными СУБД (PostgreSQL, MySQL, Oracle, MS SQL Server и др.), хотя каждая реализация имеет свои небольшие особенности.

SQL не является языком программирования в классическом смысле (алгоритмическим). Это **декларативный язык**: вы описываете, **какие** данные хотите получить, а не **как** их получить. СУБД сама решает, в каком порядке выполнять операции и какие индексы использовать.

1.4.1. Категории команд SQL

Команды SQL принято делить на несколько групп:

Группа	Назначение	Основные команды
DDL (Data Definition Language)	Определение структуры БД (создание, изменение, удаление таблиц и других объектов)	CREATE, ALTER, DROP
DML (Data Manipulation Language)	Манипулирование данными (вставка, обновление, удаление)	INSERT, UPDATE, DELETE
DQL (Data Query Language)	Выборка данных (самая часто используемая группа)	SELECT
DCL (Data Control Language)	Управление доступом (выдача и отзыв прав)	GRANT, REVOKE
TCL (Transaction Control Language)	Управление транзакциями	BEGIN, COMMIT, ROLLBACK

В рамках данного пособия мы подробно рассмотрим команды DDL, DML, DQL и TCL. DCL остаётся за рамками, так как для начального обучения он не обязателен.

1.5. Типы данных в PostgreSQL

При создании таблиц для каждого столбца нужно указать тип данных. PostgreSQL поддерживает множество типов. Ниже приведены самые употребительные.

1.5.1. Символьные типы

Тип	Описание	Пример
VARCHAR(n)	Строка переменной длины, не более n символов	VARCHAR(255)
CHAR(n)	Строка фиксированной длины, дополняется пробелами	CHAR(10)
TEXT	Строка неограниченной длины	TEXT

В большинстве случаев используют VARCHAR или TEXT. CHAR применяется редко, когда длина строки фиксирована (например, коды).

1.5.2. Числовые типы

Тип	Описание	Диапазон
SMALLINT	Целое число, 2 байта	-32768 ... 32767
INT (или INTEGER)	Целое число, 4 байта	$-2 \cdot 10^9 \dots 2 \cdot 10^9$
BIGINT	Целое число, 8 байт	очень большой
SERIAL	Автоинкрементируемое целое (псевдоним для INT с генератором)	$1 \dots 2 \cdot 10^9$
NUMERIC(p,s)	Точное число с фиксированной точкой	зависит от точности

Тип	Описание	Диапазон
REAL	Число с плавающей точкой (4 байта)	~6 десятичных цифр
DOUBLE PRECISION	Число с плавающей точкой (8 байт)	~15 цифр

Для первичных ключей часто используют SERIAL или BIGSERIAL.

1.5.3. Типы для даты и времени

Тип	Описание	Пример значения
DATE	Только дата (год-месяц-день)	'2025-03-30'
TIME	Только время	'14:30:00'
TIMESTAMP	Дата и время	'2025-03-30 14:30:00'
TIMESTAMPTZ	Дата и время с часовым поясом	'2025-03-30 14:30:00+03'

1.5.4. Логический тип

Тип	Значения
BOOLEAN	TRUE, FALSE, NULL (можно также 1/0, 'yes'/'no')

1.6. Нормализация баз данных

При проектировании базы данных важно избегать дублирования информации и аномалий (проблем при вставке, обновлении, удалении). Для этого используется **нормализация** – процесс приведения структуры БД к виду, удовлетворяющему определённым правилам (нормальным формам).

В рамках начального курса достаточно понимать **первую нормальную форму (1НФ)**:

- Каждая ячейка таблицы должна содержать атомарное (неделимое) значение.
- Не должно быть повторяющихся групп столбцов (например, телефон1, телефон2).

Также важно стремиться к тому, чтобы данные, не относящиеся напрямую к ключу, не дублировались. Обычно на практике используют **третью нормальную форму (3НФ)**.

Пример плохой структуры (не нормализовано):

читатель	книга	автор
Иванов	Война и мир	Толстой
Иванов	Анна Каренина	Толстой

Здесь информация об авторе дублируется. Лучше выделить таблицы readers, books, authors и связать их.

1.7. Учебная база данных «Library»

Во всех примерах данного пособия будет использоваться учебная база данных «**Library**», которая моделирует работу библиотеки. Она содержит следующие таблицы:

- **authors** – авторы книг.
- **books** – книги (каждая книга ссылается на автора).
- **readers** – читатели.
- **loans** – выдача книг читателям (связь многие-ко-многим).

Полное описание структуры и SQL-скрипт для создания БД приведены в **Приложении А**. Рекомендуется ознакомиться с ними перед началом занятий.

Этот раздел даёт необходимый фундамент. Если какие-то понятия остались не до конца ясными, не страшно – в следующих занятиях они будут раскрыты на конкретных примерах. Теперь можно переходить к **Разделу 2**, где начинаются практические темы, каждая из которых закрепляется задачами в боте SQLearningBot.

Раздел 2. Тематические занятия по SQL в PostgreSQL

В этом разделе вы найдёте 11 (10 тематических + итоговый проект) последовательных занятий. Каждое занятие посвящено определённой теме: от создания таблиц до сложных аналитических запросов. Теория сопровождается примерами на учебной базе данных «Library».

Как работать с разделом:

1. Внимательно изучите теорию и примеры.
 2. В конце занятия указаны номера задач, которые нужно решить в Telegram-боте SQLearningBot.
 3. Задачи внутри одного занятия нумеруются с 1. Однако они могут затрагивать материал предыдущих занятий – так вы будете повторять пройденное и глубже понимать связи между темами.
 4. Если задача не получается, используйте подсказку в боте или вернитесь к теории.
-

Занятие 1. Создание и удаление таблиц в PostgreSQL

Цель занятия: научиться создавать таблицы с учётом типов данных, первичных и внешних ключей, а также удалять таблицы и изменять их структуру.

1.1. Команда CREATE TABLE

Для создания таблицы используется команда CREATE TABLE. Общий синтаксис:

```
CREATE TABLE имя_таблицы (  
    имя_столбца тип_данных [ограничения],  
    имя_столбца тип_данных [ограничения],  
    ...  
);
```

Пример 1. Создадим таблицу authors (авторы) в нашей учебной БД «Library»:

```
CREATE TABLE authors (  
    id_author SERIAL PRIMARY KEY,  
    surname VARCHAR(100) NOT NULL,  
    name VARCHAR(100) NOT NULL,  
    patronymic VARCHAR(100),  
    birth DATE,  
    death DATE  
);
```

Разберём детали:

- SERIAL – автоинкрементируемый целочисленный тип (PostgreSQL создаёт последовательность). Удобен для первичных ключей.
- PRIMARY KEY – ограничение уникальности и непустоты.
- NOT NULL – столбец не может быть пустым.
- VARCHAR(100) – строка переменной длины до 100 символов.
- DATE – тип для хранения даты.

Пример 2. Таблица books, которая ссылается на authors:

```
CREATE TABLE books (
  id_book SERIAL PRIMARY KEY,
  name_book VARCHAR(255) NOT NULL,
  id_author INT REFERENCES authors(id_author) ON DELETE CASCADE,
  year INT CHECK (year > 0),
  pages INT DEFAULT 0
);
```

Новые элементы:

- REFERENCES authors(id_author) – внешний ключ, связывающий книгу с автором.
- ON DELETE CASCADE – при удалении автора удаляются все его книги.
- CHECK – проверка условия (год издания положительный).
- DEFAULT 0 – значение по умолчанию для поля pages.

Пример 3. Таблица readers (читатели):

```
CREATE TABLE readers (
  id_reader SERIAL PRIMARY KEY,
  surname VARCHAR(100) NOT NULL,
  name VARCHAR(100) NOT NULL,
  patronymic VARCHAR(100),
  birth DATE NOT NULL,
  registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

1.2. Команда DROP TABLE

Удаление таблицы выполняется командой DROP TABLE:

```
DROP TABLE имя_таблицы;
```

Если нужно удалить таблицу, на которую ссылаются другие таблицы (внешние ключи), используйте CASCADE:

```
DROP TABLE authors CASCADE;
```

Это удалит таблицу authors и все зависящие от неё объекты (например, таблицу books с внешним ключом).

Особенность PostgreSQL: если таблицы не существует, команда вызовет ошибку. Чтобы избежать этого, добавьте IF EXISTS:

```
DROP TABLE IF EXISTS temp_table;
```

1.3. Изменение структуры таблицы (ALTER TABLE)

После создания таблицы можно добавлять, удалять или изменять столбцы.

Добавление столбца:

```
ALTER TABLE books ADD COLUMN publisher VARCHAR(200);
```

Удаление столбца:

```
ALTER TABLE books DROP COLUMN publisher;
```

Изменение типа столбца:

```
ALTER TABLE books ALTER COLUMN year TYPE SMALLINT;
```

Добавление ограничения (например, внешнего ключа):

```
ALTER TABLE books ADD CONSTRAINT fk_author FOREIGN KEY (id_author) REFERENCES authors(id_author);
```

1.4. Особенности PostgreSQL при создании таблиц

- **Схемы (schemas).** По умолчанию таблицы создаются в схеме public. Можно создавать свои схемы: CREATE SCHEMA school; CREATE TABLE school.students (...);
- **Временные таблицы.** Существуют только в рамках сессии: CREATE TEMP TABLE temp_data (id INT);
- **Наследование таблиц.** PostgreSQL поддерживает наследование: одна таблица может быть «дочерней» для другой. Эта возможность выходит за рамки начального курса.

1.5. Практический пример: создание всех трёх таблиц для «Library»

```
-- Таблица авторов
CREATE TABLE authors (
  id_author SERIAL PRIMARY KEY,
  surname VARCHAR(100) NOT NULL,
  name VARCHAR(100) NOT NULL,
  patronymic VARCHAR(100),
  birth DATE,
  death DATE
);

-- Таблица книг
CREATE TABLE books (
  id_book SERIAL PRIMARY KEY,
  name_book VARCHAR(255) NOT NULL,
  id_author INT REFERENCES authors(id_author) ON DELETE CASCADE,
  year INT CHECK (year > 0),
  pages INT DEFAULT 0
);
```

```
);

-- Таблица читателей
CREATE TABLE readers (
  id_reader SERIAL PRIMARY KEY,
  surname VARCHAR(100) NOT NULL,
  name VARCHAR(100) NOT NULL,
  patronymic VARCHAR(100),
  birth DATE NOT NULL,
  registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

После выполнения этих команд база данных «Library» получит структуру, на которой будут строиться все последующие примеры.

Задачи для закрепления (Занятие 1)

Решите следующие задачи в Telegram-боте SQLearningBot. Они проверят ваше понимание материала этого занятия.

1. **Создание таблицы.** Напишите команду CREATE TABLE для таблицы publishers(издательства) со следующими полями:
 - id_publisher – первичный ключ, автоинкремент,
 - name – название издательства, строка до 200 символов, не пустое,
 - city – город, строка до 100 символов,
 - founded_year – год основания, целое число (проверка, что год не меньше 1500 и не больше текущего года (текущий год укажите статически)).
2. **Добавление столбца.** Добавьте в таблицу books столбец isbn типа VARCHAR(13)(международный стандартный книжный номер).
3. **Удаление столбца.** Удалите из таблицы books столбец pages, который был добавлен в примере.
4. **Изменение типа.** В таблице readers измените тип столбца registered_at с TIMESTAMP на DATE (дата без времени). Подсказка: используйте ALTER TABLE ... ALTER COLUMN ... TYPE DATE;
5. **Создание таблицы со связью.** Создайте таблицу loans (выдача книг читателям), которая содержит:
 - id_loan – первичный ключ,
 - id_book – внешний ключ на books(id_book),
 - id_reader – внешний ключ на readers(id_reader),
 - loan_date – дата выдачи, тип DATE (по умолчанию текущая дата),
 - return_date – дата возврата, может быть пустой.
 Используйте ON DELETE CASCADE для внешних ключей.

6. **Удаление таблицы с зависимостями.** Напишите команду, которая удаляет таблицу `publishers` вместе со всеми объектами, которые на неё ссылаются (если бы такие были). Используйте `CASCADE`.
 7. **Создание временной таблицы.** Создайте временную таблицу `temp_books` с одним столбцом `temp_name` типа `TEXT`. Временная таблица должна автоматически удалиться после завершения сессии.
-

После успешного решения этих задач вы сможете переходить к следующему занятию, где мы будем заполнять таблицы данными и редактировать их. Удачи!

Занятие 2. Вставка, обновление и удаление данных (INSERT, UPDATE, DELETE)

Цель занятия: научиться добавлять новые строки в таблицы, изменять существующие данные и удалять записи с помощью операторов DML (INSERT, UPDATE, DELETE).

2.1. Вставка данных: команда INSERT INTO

После создания таблиц их необходимо заполнить данными. Основная команда для вставки – INSERT INTO.

Базовый синтаксис (указываются все столбцы по порядку):

```
INSERT INTO имя_таблицы VALUES (значение1, значение2, ...);
```

Рекомендуемый синтаксис (явно перечисляются столбцы):

```
INSERT INTO имя_таблицы (столбец1, столбец2, ...) VALUES (значение1, значение2, ...);
```

Второй вариант предпочтительнее, так как он не зависит от порядка столбцов в таблице и позволяет опустить столбцы со значениями по умолчанию (например, SERIAL).

Пример 1. Добавим автора в таблицу authors:

```
INSERT INTO authors (surname, name, patronymic, birth, death)
VALUES ('Пушкин', 'Александр', 'Сергеевич', '1799-06-06', '1837-02-10');
```

Обратите внимание:

- id_author не указан – он заполнится автоматически из последовательности SERIAL.
- Строковые значения и даты заключаются в одинарные кавычки.
- Числа (если бы были) – без кавычек.

Пример 2. Добавим книгу, связанную с автором (предположим, у Пушкина id_author = 1):

```
INSERT INTO books (name_book, id_author, year, pages)
VALUES ('Евгений Онегин', 1, 1833, 480);
```

Пример 3. Вставка нескольких строк за один запрос:

```
INSERT INTO books (name_book, id_author, year, pages) VALUES
('Капитанская дочка', 1, 1836, 320),
('Руслан и Людмила', 1, 1820, 280);
```

Особенность PostgreSQL: можно использовать RETURNING, чтобы сразу получить сгенерированные значения (например, id_book):

```
INSERT INTO authors (surname, name) VALUES ('Толстой', 'Лев') RETURNING id_author;
```

2.2. Обновление данных: команда UPDATE

Для изменения существующих строк используется UPDATE.

Синтаксис:

```
UPDATE имя_таблицы SET столбец1 = новое_значение1, столбец2 = новое_значение2, ... WHERE условие;
```

Если опустить WHERE, обновятся **все** строки таблицы – действуйте осторожно!

Пример 1. Исправим год издания книги «Руслан и Людмила» (допустим, правильный год – 1820, он уже указан, а мы для примера изменим на 1821, потом вернём):

```
UPDATE books SET year = 1821 WHERE name_book = 'Руслан и Людмила';
```

Пример 2. Добавим отчество автору, у которого оно не заполнено:

```
UPDATE authors SET patronymic = 'Николаевич' WHERE surname = 'Толстой' AND name = 'Лев';
```

Пример 3. Увеличим количество страниц у всех книг, изданных после 1830 года, на 10:

```
UPDATE books SET pages = pages + 10 WHERE year > 1830;
```

Особенность PostgreSQL: можно обновлять данные, используя значения из других таблиц (через FROM), но это будет рассмотрено позже.

2.3. Удаление данных: команда DELETE

Для удаления строк используется DELETE.

Синтаксис:

```
DELETE FROM имя_таблицы WHERE условие;
```

Если WHERE отсутствует, удаляются **все** строки таблицы (но сама таблица остаётся).

Пример 1. Удалим книгу «Капитанская дочка»:

```
DELETE FROM books WHERE name_book = 'Капитанская дочка';
```

Пример 2. Удалим всех авторов, у которых нет ни одной книги (предположим, что такие есть):

```
DELETE FROM authors WHERE id_author NOT IN (SELECT DISTINCT id_author FROM books);
```

Ключевое слово DISTINCT удаляет дубликаты id_author из подзапроса, делая запрос более эффективным и понятным. Без него запрос также будет работать, но при большом количестве книг может выполняться дольше.

Внимание: из-за внешнего ключа с ON DELETE CASCADE при удалении автора удалятся и его книги. Если вы хотите сохранить книги, не используйте CASCADE или сначала переназначьте id_author книгам.

2.4. Команда TRUNCATE

Если нужно быстро удалить **все строки** из таблицы (быстрее, чем DELETE без условия), используют TRUNCATE:

```
TRUNCATE TABLE имя_таблицы;
```

TRUNCATE нельзя откатить в некоторых СУБД (в PostgreSQL откатить можно, если команда внутри транзакции). Он сбрасывает последовательности (SERIAL) и не срабатывает для триггеров DELETE.

2.5. Практический пример: наполнение базы «Library» тестовыми данными

```
-- Добавляем авторов
INSERT INTO authors (surname, name, patronymic, birth, death) VALUES
('Пушкин', 'Александр', 'Сергеевич', '1799-06-06', '1837-02-10'),
('Толстой', 'Лев', 'Николаевич', '1828-09-09', '1910-11-20'),
('Достоевский', 'Фёдор', 'Михайлович', '1821-11-11', '1881-02-09');

-- Добавляем книги (id_author: 1 - Пушкин, 2 - Толстой, 3 - Достоевский)
INSERT INTO books (name_book, id_author, year, pages) VALUES
('Евгений Онегин', 1, 1833, 480),
('Капитанская дочка', 1, 1836, 320),
('Война и мир', 2, 1869, 1225),
('Анна Каренина', 2, 1877, 864),
('Преступление и наказание', 3, 1866, 672),
('Идиот', 3, 1869, 640);

-- Добавляем читателей
INSERT INTO readers (surname, name, patronymic, birth) VALUES
('Иванов', 'Иван', 'Иванович', '1990-05-15'),
('Петрова', 'Мария', 'Сергеевна', '1995-11-23');
```

После выполнения этих команд база данных «Library» будет содержать начальные данные, с которыми можно работать в следующих занятиях.

Задачи для закрепления (Занятие 2)

Решите следующие задачи в боте SQLearningBot. Они проверят умение вставлять, обновлять и удалять данные.

1. **Вставка одного автора.** Добавьте в таблицу authors автора с фамилией «Гоголь», именем «Николай», отчеством «Васильевич». Год рождения – 1809, год смерти – 1852. Используйте явный перечень столбцов.
2. **Вставка нескольких книг.** Добавьте в таблицу books три книги автора с id_author = 1(Пушкин):

- «Сказка о царе Салтане» (год 1831, страниц 120)
- «Медный всадник» (год 1833, страниц 80)
- «Борис Годунов» (год 1831, страниц 150)

Напишите один запрос INSERT, который добавляет все три книги.

3. **Обновление данных (с условием).** У книги «Война и мир» измените количество страниц на 1300. Используйте WHERE по названию книги.
4. **Обновление всех строк (осторожно!).** Напишите запрос, который увеличивает год издания всех книг на 1. **Внимание:** в реальной базе так делать не стоит, но для тренировки выполните. Затем исправьте обратно (уменьшите на 1).
5. **Удаление по условию.** Удалите из таблицы books все книги, у которых количество страниц меньше 100.
6. **Удаление с подзапросом.** Удалите из таблицы authors всех авторов, у которых нет книг (используйте NOT IN или NOT EXISTS).
7. **Использование RETURNING.** Добавьте нового читателя с фамилией «Сидоров», именем «Алексей», отчеством «Петрович», датой рождения «2000-01-01» и верните его сгенерированный id_reader одним запросом.

После успешного выполнения этих задач вы сможете переходить к занятию 3, где начнём осваивать выборку данных (SELECT).

Занятие 3. Выборка данных: команда SELECT

Цель занятия: освоить базовую выборку данных из таблиц с помощью команды SELECT, научиться фильтровать строки, исключать дубликаты и выполнять простые вычисления.

3.1. Простейшая выборка: SELECT ... FROM

Самая частая операция в SQL – получение данных из таблиц. Для этого используется команда SELECT.

Синтаксис:

SELECT столбцы **FROM** имя_таблицы;

- * (звёздочка) – выбрать все столбцы.
- Перечисление столбцов через запятую – выбрать только указанные столбцы.

Пример 1. Вывести все данные из таблицы books:

```
SELECT * FROM books;
```

Пример 2. Вывести только названия и год издания книг:

```
SELECT name_book, year FROM books;
```

Пример 3. Вывести фамилии и имена всех авторов:

```
SELECT surname, name FROM authors;
```

3.2. Устранение дубликатов: DISTINCT

Иногда в столбце встречаются повторяющиеся значения. Чтобы получить уникальные значения, используется ключевое слово DISTINCT.

Синтаксис:

SELECT DISTINCT столбец **FROM** имя_таблицы;

Пример. Вывести уникальные годы издания книг:

```
SELECT DISTINCT year FROM books ORDER BY year;
```

3.3. Фильтрация строк: WHERE

Чтобы выбрать не все строки, а только те, которые удовлетворяют условию, используется WHERE.

Синтаксис:

SELECT столбцы **FROM** имя_таблицы **WHERE** условие;

Операторы сравнения: =, <> (не равно), >, <, >=, <=.

Логические операторы: AND, OR, NOT.

Пример 1. Вывести книги, изданные после 1860 года:

```
SELECT name_book, year FROM books WHERE year > 1860;
```

Пример 2. Вывести книги автора с id_author = 2 (Толстой), изданные не ранее 1870 года:

```
SELECT name_book, year FROM books WHERE id_author = 2 AND year >= 1870;
```

Пример 3. Вывести авторов, у которых нет отчества (patronymic равно NULL):

```
SELECT surname, name FROM authors WHERE patronymic IS NULL;
```

Важно: NULL – это отсутствие значения, его нельзя проверить оператором =. Используйте IS NULL или IS NOT NULL.

3.4. Проверка вхождения: IN и BETWEEN

Для удобной проверки набора значений используется IN, для диапазона – BETWEEN.

Пример 1. Вывести книги авторов с id_author 1 или 3:

```
SELECT name_book, id_author FROM books WHERE id_author IN (1, 3);
```

Пример 2. Вывести книги, изданные между 1830 и 1850 годами включительно:

```
SELECT name_book, year FROM books WHERE year BETWEEN 1830 AND 1850;
```

3.5. Поиск по шаблону: LIKE

Для поиска по части строки используется оператор LIKE с символами-шаблонами:

- % – любая последовательность символов (в том числе пустая).
- _ – ровно один любой символ.

Пример 1. Найти книги, название которых начинается со слова «Война»:

```
SELECT name_book FROM books WHERE name_book LIKE 'Война%';
```

Пример 2. Найти авторов, у которых фамилия оканчивается на «ий»:

```
SELECT surname FROM authors WHERE surname LIKE '%ий';
```

Пример 3. Найти читателей, у которых имя состоит из 4 букв (четыре любых символа):

```
SELECT name FROM readers WHERE name LIKE '___';
```

Особенность PostgreSQL: по умолчанию LIKE чувствителен к регистру. Для нечувствительного поиска используйте ILIKE:

```
SELECT name_book FROM books WHERE name_book ILIKE 'война%';
```

3.6. Вычисления в SELECT

В запросе можно выполнять арифметические операции, конкатенацию строк, вызывать функции.

Пример 1. Вывести название книги и количество страниц, увеличенное на 50 (без изменения данных):

```
SELECT name_book, pages + 50 AS pages_plus_50 FROM books;
```

Ключевое слово AS задаёт псевдоним для столбца (необязательно, но улучшает читаемость).

Пример 2. Вывести полное имя автора (фамилию, имя и отчество) одной строкой:

```
SELECT surname || ' ' || name || ' ' || COALESCE(patronymic, '') AS full_name FROM authors;
```

|| – конкатенация строк в PostgreSQL. COALESCE заменяет NULL на пустую строку.

3.7. Практический пример: различные запросы к «Library»

-- 1. Все книги Пушкина (id_author = 1)

```
SELECT name_book, year FROM books WHERE id_author = 1;
```

-- 2. Уникальные года рождения читателей

```
SELECT DISTINCT birth FROM readers;
```

-- 3. Книги, в названии которых есть слово «мир» (без учёта регистра)

```
SELECT name_book FROM books WHERE name_book ILIKE '%мир%';
```

-- 4. Вывести авторов, родившихся в XIX веке (1801–1900)

```
SELECT surname, name, birth FROM authors WHERE birth BETWEEN '1801-01-01' AND '1900-12-31';
```

-- 5. Для каждой книги вывести название и количество страниц, округлённое до сотен (в большую сторону)

```
SELECT name_book, CEIL(pages / 100.0) * 100 AS pages_rounded_up FROM books;
```

Задачи для закрепления (Занятие 3)

Решите следующие задачи в боте SQLearningBot. Они проверят умение составлять запросы на выборку данных.

1. **Выборка всех столбцов.** Напишите запрос, который выводит все данные из таблицы `readers`.
2. **Выборка конкретных столбцов.** Выведите фамилию, имя и дату рождения всех читателей.
3. **Устранение дубликатов.** Выведите уникальные годы рождения читателей. Отсортируйте результат по возрастанию года.
4. **Фильтрация по условию (один оператор).** Выведите названия книг, у которых количество страниц больше 500.
5. **Фильтрация по условию (два оператора).** Выведите названия и годы издания книг, написанных автором с `id_author = 2` (Толстой), но изданных не ранее 1870 года.
6. **Проверка на NULL.** Выведите всех авторов, у которых заполнена дата смерти (`death IS NOT NULL`).
7. **Использование IN.** Выведите названия книг, у которых `id_author` равен 1 или 3.
8. **Использование BETWEEN.** Выведите книги, изданные в период с 1830 по 1850 год включительно.
9. **Поиск по шаблону (LIKE).** Выведите фамилии авторов, которые начинаются на букву «Д» (без учёта регистра, используйте `ILIKE`).
10. **Вычисления и псевдонимы.** Выведите название книги и количество страниц, уменьшенное на 10% (как `pages_reduced`). Результат округлите до целого числа.
11. **Конкатенация строк.** Выведите полное имя читателя (фамилию, имя и отчество) в одном столбце с псевдонимом `full_name`. Если отчество отсутствует (`NULL`), выведите только фамилию и имя.
12. **Комбинированный запрос.** Выведите уникальные фамилии авторов, у которых год рождения позже 1800 года, отсортированные по фамилии в алфавитном порядке.

После успешного выполнения этих задач вы научитесь получать данные из базы в любых нужных комбинациях. В следующем занятии разберём сортировку (`ORDER BY`) и ограничение количества записей (`LIMIT`).

Занятие 4. Сортировка и ограничение количества записей (ORDER BY, LIMIT)

Цель занятия: научиться упорядочивать результаты запросов, а также ограничивать количество выводимых строк.

4.1. Сортировка: ORDER BY

По умолчанию строки в результате запроса выводятся в том порядке, в котором они были добавлены в таблицу (или в порядке использования индексов). Чтобы задать явный порядок, используется ORDER BY.

Синтаксис:

```
SELECT столбцы FROM имя_таблицы ORDER BY столбец [ASC | DESC];
```

- ASC (по умолчанию) – сортировка по возрастанию (от меньшего к большему, от А до Я).
- DESC – сортировка по убыванию.

Пример 1. Вывести книги в порядке возрастания года издания:

```
SELECT name_book, year FROM books ORDER BY year ASC;
```

или просто ORDER BY year (ASC подразумевается).

Пример 2. Вывести книги, отсортированные по убыванию количества страниц:

```
SELECT name_book, pages FROM books ORDER BY pages DESC;
```

Пример 3. Сортировка по нескольким столбцам (сначала по автору, затем по году внутри каждого автора):

```
SELECT name_book, id_author, year FROM books ORDER BY id_author, year;
```

Пример 4. Вывести авторов, отсортированных по фамилии, а при одинаковой фамилии – по имени:

```
SELECT surname, name FROM authors ORDER BY surname, name;
```

Особенность PostgreSQL: NULL при сортировке по умолчанию считается больше любого значения (NULLS LAST при ASC, NULLS FIRST при DESC). Поведение можно изменить явно:

```
SELECT surname, name, patronymic FROM authors ORDER BY patronymic NULLS FIRST;
```

4.2. Ограничение количества записей: LIMIT

Если нужно получить не все строки, а только определённое количество (например, первые 10), используется LIMIT.

Синтаксис:

```
SELECT столбцы FROM имя_таблицы LIMIT количество_строк;
```

Пример 1. Вывести первые 3 книги из таблицы books:

```
SELECT * FROM books LIMIT 3;
```

Пример 2. Вывести трёх самых молодых читателей (отсортировав по дате рождения по убыванию):

```
SELECT surname, name, birth FROM readers ORDER BY birth DESC LIMIT 3;
```

Пример 3. Вывести 5 самых старых книг (по году издания):

```
SELECT name_book, year FROM books ORDER BY year LIMIT 5;
```

4.3. Пропуск записей: OFFSET

OFFSET позволяет пропустить указанное количество строк перед тем, как начать вывод.

Синтаксис:

```
SELECT столбцы FROM имя_таблицы OFFSET N LIMIT M;
```

Сначала пропускается N строк, затем выводятся следующие M строк.

Пример 1. Пропустить первые 2 книги и вывести следующие 3 (постраничный вывод, страница 2 по 3 записи):

```
SELECT * FROM books ORDER BY id_book OFFSET 2 LIMIT 3;
```

Пример 2. Вывести книги, начиная с 4-й по 6-ю (включительно), отсортированные по году:

```
SELECT name_book, year FROM books ORDER BY year OFFSET 3 LIMIT 3;
```

4.4. Практический пример: комбинирование ORDER BY, LIMIT и OFFSET

```
-- 1. Самые новые книги (первые 3 по убыванию года)
SELECT name_book, year FROM books ORDER BY year DESC LIMIT 3;

-- 2. Две самые длинные книги (по количеству страниц)
SELECT name_book, pages FROM books ORDER BY pages DESC LIMIT 2;

-- 3. Вторая страница списка авторов (по 5 авторов на странице)
-- Страница 1: OFFSET 0 LIMIT 5
-- Страница 2: OFFSET 5 LIMIT 5
SELECT surname, name FROM authors ORDER BY surname LIMIT 5 OFFSET 5;

-- 4. Три самых молодых читателя (самые поздние даты рождения)
SELECT surname, name, birth FROM readers ORDER BY birth DESC LIMIT 3;
```

Особенность PostgreSQL: если используется LIMIT, рекомендуется всегда указывать ORDER BY, иначе результат может быть непредсказуемым (какие именно строки попадут в ограничение, не определено).

4.5. Альтернативы и стандарт SQL

В стандарте SQL вместо LIMIT иногда используется FETCH FIRST n ROWS ONLY. PostgreSQL поддерживает оба варианта.

```
SELECT name_book FROM books ORDER BY year FETCH FIRST 3 ROWS ONLY;
```

Это эквивалентно LIMIT 3. Однако LIMIT короче и чаще используется на практике.

Задачи для закрепления (Занятие 4)

Решите следующие задачи в боте SQLearningBot. Они проверят умение сортировать и ограничивать выборку.

1. **Простая сортировка по возрастанию.** Выведите названия книг и годы издания, отсортированные по году (от старых к новым).
2. **Сортировка по убыванию.** Выведите фамилии, имена и даты рождения читателей, отсортированные по дате рождения (самые молодые – первые).
3. **Сортировка по двум полям.** Выведите названия книг, id автора и год издания, отсортированные сначала по id автора (по возрастанию), а внутри одного автора – по году (по убыванию).
4. **Сортировка с NULL.** Выведите фамилию, имя и отчество авторов, отсортированные по отчеству так, чтобы NULL шли в начале списка.
5. **Простой LIMIT.** Выведите первые 5 записей из таблицы books (без сортировки, но для определённости используйте ORDER BY id_book).
6. **LIMIT с сортировкой.** Выведите 3 книги с наибольшим количеством страниц. Результат должен содержать название и количество страниц.
7. **Пропуск записей (OFFSET).** Пропустите первые 2 книги и выведите следующие 4 книги, отсортированные по названию.
8. **Постраничный вывод.** Напишите запрос, который выводит вторую страницу списка авторов (по 3 автора на странице), отсортированных по фамилии. Результат должен содержать фамилию и имя.
9. **Комбинированный запрос (фильтрация + сортировка + LIMIT).** Выведите названия книг, изданных после 1850 года, отсортированные по году (от новых к старым), и возьмите только первые 2 записи.
10. **Использование FETCH FIRST.** Перепишите задачу 6 с использованием стандартного синтаксиса FETCH FIRST 3 ROWS ONLY.
11. **Сложная сортировка с вычислениями.** Выведите название книги и количество страниц, округлённое до десятков (в меньшую сторону).

Отсортируйте результат по этому округлённому значению по убыванию, а затем по названию.

12. **Проверка на прочность.** Выведите уникальные фамилии авторов, у которых год рождения позже 1800 года, отсортированные по длине фамилии (от самых коротких к самым длинным). Если длины совпадают – по алфавиту.

После выполнения этих задач вы будете уверенно управлять порядком и объёмом выводимых данных. В следующем занятии перейдём к агрегатным функциям и группировке (COUNT, SUM, AVG, GROUP BY).

Занятие 5. Агрегатные функции и группировка (COUNT, SUM, AVG, GROUP BY, HAVING)

Цель занятия: научиться вычислять итоговые значения по группам строк: количество записей, сумму, среднее, минимум, максимум, а также использовать группировку и фильтрацию групп.

5.1. Что такое агрегатные функции

Агрегатные функции обрабатывают набор строк и возвращают одно итоговое значение. Чаще всего они используются без группировки (тогда результат – одна строка для всей таблицы) или с GROUP BY (тогда результат – по одной строке на каждую группу).

Основные агрегатные функции в PostgreSQL:

Функция	Назначение
COUNT(*)	количество строк (включая NULL)
COUNT(столбец)	количество непустых значений в столбце
SUM(столбец)	сумма значений (для числовых столбцов)
AVG(столбец)	среднее арифметическое
MIN(столбец)	минимальное значение
MAX(столбец)	максимальное значение

Пример 1. Общее количество книг в таблице books:

```
SELECT COUNT(*) AS total_books FROM books;
```

Пример 2. Количество книг, у которых указано количество страниц (не NULL):

```
SELECT COUNT(pages) AS books_with_pages FROM books;
```

Пример 3. Общее количество страниц во всех книгах:

```
SELECT SUM(pages) AS total_pages FROM books;
```

Пример 4. Среднее количество страниц:

```
SELECT AVG(pages) AS avg_pages FROM books;
```

Пример 5. Самая старая и самая новая книга (по году издания):

```
SELECT MIN(year) AS oldest_year, MAX(year) AS newest_year FROM books;
```

5.2. Группировка: GROUP BY

Часто нужно получить итоги не по всей таблице, а по группам (например, количество книг каждого автора). Для этого используется GROUP BY.

Синтаксис:

```
SELECT столбец_группировки, агрегатная_функция(столбец)
FROM таблица
GROUP BY столбец_группировки;
```

Пример 1. Количество книг каждого автора:

```
SELECT id_author, COUNT(*) AS book_count
FROM books
GROUP BY id_author;
```

Пример 2. Для каждого автора – средний год издания его книг:

```
SELECT id_author, AVG(year) AS avg_year
FROM books
GROUP BY id_author;
```

Пример 3. Группировка по нескольким столбцам: количество книг по каждому году издания и каждому автору:

```
SELECT id_author, year, COUNT(*) AS cnt
FROM books
GROUP BY id_author, year
ORDER BY id_author, year;
```

Важно: в SELECT можно включать только столбцы, перечисленные в GROUP BY, или агрегатные функции. Любой другой столбец вызовет ошибку.

5.3. Фильтрация групп: HAVING

WHERE фильтрует строки до группировки. HAVING фильтрует группы после группировки.

Синтаксис:

```
SELECT столбец_группировки, агрегатная_функция(столбец)
FROM таблица
GROUP BY столбец_группировки
HAVING условие_для_группы;
```

Пример 1. Вывести только тех авторов, у которых больше 2 книг:

```
SELECT id_author, COUNT(*) AS book_count
FROM books
GROUP BY id_author
HAVING COUNT(*) > 2;
```

Пример 2. Вывести авторов, у которых средний год издания книг позже 1860 года:

```
SELECT id_author, AVG(year) AS avg_year
FROM books
GROUP BY id_author
HAVING AVG(year) > 1860;
```

Пример 3. Комбинация WHERE и HAVING: сначала отфильтровать книги, изданные после 1830 года, затем сгруппировать по авторам и оставить только тех, у которых суммарное количество страниц больше 1000:

```
SELECT id_author, SUM(pages) AS total_pages
FROM books
WHERE year > 1830
GROUP BY id_author
HAVING SUM(pages) > 1000;
```

5.4. Практический пример: аналитические запросы к «Library»

```
-- 1. Общее количество авторов
SELECT COUNT(*) FROM authors;

-- 2. Количество книг каждого автора (с выводом фамилии через JOIN – будет в занятии 6)
-- пока только с id_author
SELECT id_author, COUNT(*) FROM books GROUP BY id_author;

-- 3. Максимальное количество страниц среди книг Толстого (id_author = 2)
SELECT MAX(pages) FROM books WHERE id_author = 2;

-- 4. Среднее количество страниц по каждому автору, у которого среднее больше 500
SELECT id_author, AVG(pages) AS avg_pages
FROM books
GROUP BY id_author
HAVING AVG(pages) > 500;

-- 5. Количество книг по годам издания, от самых старых к новым
SELECT year, COUNT(*) AS cnt
FROM books
GROUP BY year
ORDER BY year;
```

5.5. Особенности PostgreSQL

- COUNT(*) учитывает все строки, включая те, где все столбцы NULL.
- SUM, AVG игнорируют NULL.
- AVG возвращает числовой тип, зависящий от входных данных. Для целых чисел результат будет numeric.
- Можно использовать агрегатные функции с DISTINCT: COUNT(DISTINCT id_author) – количество уникальных авторов в таблице книг.
- PostgreSQL поддерживает также статистические агрегаты (например, stddev, variance), но они выходят за рамки начального курса.

Задачи для закрепления (Занятие 5)

Решите следующие задачи в боте SQLearningBot. Они проверят понимание агрегатных функций и группировки.

1. **Общее количество читателей.** Подсчитайте, сколько читателей зарегистрировано в таблице `readers`.
2. **Количество книг без страниц.** Подсчитайте, сколько книг не имеют указанного количества страниц (`pages IS NULL`).
3. **Суммарное количество страниц.** Найдите общее количество страниц во всех книгах.
4. **Средний год издания книг.** Вычислите средний год издания книг в таблице `books`. Результат округлите до целого числа.
5. **Самая толстая и самая тонкая книга.** Найдите максимальное и минимальное количество страниц среди всех книг.
6. **Количество книг по авторам.** Выведите `id_author` и количество книг каждого автора. Отсортируйте по убыванию количества.
7. **Средний год издания по авторам.** Выведите `id_author` и средний год издания его книг. Округлите среднее до целого.
8. **Фильтрация групп (HAVING).** Выведите `id_author`, у которых общее количество страниц (`SUM(pages)`) больше 1000. Отсортируйте по убыванию суммы.
9. **Комбинация WHERE и HAVING.** Выведите `id_author` и количество книг, изданных после 1830 года, но только для тех авторов, у которых таких книг больше 1.
10. **Использование COUNT(DISTINCT).** Подсчитайте, сколько уникальных годов издания представлено в таблице `books`.
11. **Сложная агрегация.** Выведите максимальное, минимальное и среднее количество страниц для книг, изданных в XIX веке (1801–1900). Назовите столбцы `max_pages`, `min_pages`, `avg_pages`.
12. **Группировка с вычислением.** Для каждого года издания выведите количество книг, общее количество страниц и среднее количество страниц. Отсортируйте по году.

После выполнения этих задач вы сможете легко подводить итоги по любым наборам данных. В следующем занятии разберём объединение таблиц (`JOIN`), которое позволит выводить данные из нескольких связанных таблиц в одном запросе (например, название книги вместе с фамилией автора).

Занятие 6. Объединение таблиц (JOIN)

Цель занятия: научиться объединять данные из двух и более таблиц по ключам, чтобы получать полную информацию (например, название книги вместе с фамилией автора).

6.1. Зачем нужны JOIN

В реляционных базах данных информация часто распределена по нескольким связанным таблицам, чтобы избежать дублирования. Например, в таблице books хранится id_author(ссылка на автора), а фамилия и имя автора – в таблице authors. Чтобы вывести список книг с фамилиями авторов, нужно объединить эти две таблицы.

6.2. Типы JOIN

Самые распространённые типы объединений:

Тип JOIN	Что возвращает
INNER JOIN	только строки, для которых найдено соответствие в обеих таблицах
LEFT JOIN	все строки из левой таблицы + соответствующие строки из правой (если нет соответствия – NULL)
RIGHT JOIN	все строки из правой таблицы + соответствующие из левой (аналог LEFT JOIN, но таблицы меняются местами)
FULL JOIN	все строки из обеих таблиц, где есть соответствие, и NULL с той стороны, где соответствия нет

На практике чаще всего используются INNER JOIN и LEFT JOIN.

6.3. Синтаксис JOIN

```
SELECT столбцы
FROM таблица1
[INNER | LEFT | RIGHT | FULL] JOIN таблица2
ON таблица1.ключ = таблица2.ключ;
```

Можно использовать псевдонимы таблиц для краткости:

```
SELECT b.name_book, a.surname
FROM books AS b
JOIN authors AS a ON b.id_author = a.id_author;
```

6.4. Примеры JOIN на базе «Library»

Пример 1. INNER JOIN – книги с авторами (только те, у которых автор существует):

```
SELECT books.name_book, authors.surname, authors.name
FROM books
INNER JOIN authors ON books.id_author = authors.id_author;
```

Пример 2. LEFT JOIN – все книги, даже если автор не указан (id_author = NULL):

```
SELECT books.name_book, authors.surname
FROM books
LEFT JOIN authors ON books.id_author = authors.id_author;
```

Если у какой-то книги id_author равен NULL, в колонке surname будет NULL.

Пример 3. Псевдонимы таблиц (сокращённая запись):

```
SELECT b.name_book, a.surname
FROM books b
JOIN authors a ON b.id_author = a.id_author;
```

Пример 4. Объединение трёх таблиц (книги → авторы → читатели через таблицу loans, если бы она была).

Пока loans нет, но можно показать на будущее:

```
SELECT r.surname, b.name_book
FROM readers r
JOIN loans l ON r.id_reader = l.id_reader
JOIN books b ON l.id_book = b.id_book;
```

6.5. JOIN с агрегатными функциями

Объединения часто используются вместе с группировкой.

Пример 5. Количество книг по каждому автору (с фамилией, а не id_author):

```
SELECT a.surname, COUNT(b.id_book) AS book_count
FROM authors a
LEFT JOIN books b ON a.id_author = b.id_author
GROUP BY a.id_author, a.surname;
```

LEFT JOIN гарантирует, что авторы без книг тоже попадут в результат (с нулём).

Пример 6. Для каждого автора – средний год издания его книг (только у тех, у кого есть книги):

```
SELECT a.surname, ROUND(AVG(b.year)) AS avg_year
FROM authors a
JOIN books b ON a.id_author = b.id_author
GROUP BY a.id_author, a.surname;
```

6.6. Особенности PostgreSQL

- JOIN без указания типа означает INNER JOIN.
- Можно использовать USING, если имена столбцов для соединения одинаковы:
JOIN authors USING (id_author) вместо ON books.id_author = authors.id_author.
- PostgreSQL умеет оптимизировать запросы с JOIN, используя индексы по внешним ключам (рекомендуется создавать индексы на столбцы, по которым часто делаются соединения).

6.7. Практический пример: несколько JOIN

```
-- 1. Список книг с фамилиями авторов и годами издания, отсортированный по фамилии
SELECT b.name_book, a.surname, b.year
FROM books b
JOIN authors a ON b.id_author = a.id_author
ORDER BY a.surname;

-- 2. Авторы и количество их книг (включая авторов без книг)
SELECT a.surname, COUNT(b.id_book) AS cnt
FROM authors a
LEFT JOIN books b ON a.id_author = b.id_author
GROUP BY a.id_author, a.surname
ORDER BY cnt DESC;

-- 3. Книги, у которых количество страниц больше среднего (подзапрос будет в занятии 7)
-- пока без подзапроса, но с JOIN можно сравнить с фиксированным числом
SELECT b.name_book, b.pages, a.surname
FROM books b
JOIN authors a ON b.id_author = a.id_author
WHERE b.pages > 500;
```

Задачи для закрепления (Занятие 6)

Решите следующие задачи в боте SQLearningBot. Они проверят умение объединять таблицы.

1. **INNER JOIN (книги + авторы).** Выведите название книги, фамилию и имя автора для всех книг, у которых указан автор. Используйте INNER JOIN.
2. **LEFT JOIN (авторы + книги).** Выведите фамилию автора и название его книги. Включите всех авторов, даже если у них нет книг (тогда название книги будет NULL).
3. **JOIN с сортировкой.** Выведите название книги, фамилию автора и год издания, отсортированные по фамилии автора, а затем по году (от старых к новым).
4. **JOIN с агрегацией (количество книг по авторам).** Выведите фамилию автора и количество написанных им книг. Используйте JOIN и GROUP BY. Авторов без книг не выводить.

5. **LEFT JOIN с агрегацией (включая авторов без книг).** Выведите фамилию автора и количество книг. Если у автора нет книг, количество должно быть 0.
6. **JOIN с фильтрацией.** Выведите названия книг и фамилии авторов, у которых год издания книги позже 1850 года.
7. **Объединение трёх таблиц (через loans).**
Примечание: перед решением этой задачи убедитесь, что таблица loans создана (Занятие 1, задача 5). Если нет – создайте её. Выведите фамилию читателя, название книги и дату выдачи для всех записей в таблице loans. Используйте два JOIN.
8. **JOIN с вычисляемым столбцом.** Выведите название книги, фамилию автора и количество страниц, увеличенное на 10% (как `pages_plus_percent`). Округлите до целого.
9. **Группировка с JOIN и HAVING.** Выведите фамилию автора и средний год издания его книг, но только для тех авторов, у которых средний год позже 1850 года.
10. **Самый плодовитый автор.** Выведите фамилию автора и количество его книг, отсортируйте по убыванию количества и оставьте только одного (самого плодовитого). Используйте JOIN, GROUP BY, ORDER BY и LIMIT.
11. **Книги без авторов (проверка ссылочной целостности).** Выведите все книги, у которых `id_author` не соответствует ни одному `id_author` в таблице `authors` (например, если книга ссылается на несуществующего автора). Используйте LEFT JOIN и проверку WHERE `authors.id_author IS NULL`.
12. **Полная информация о книге (автор + если есть читатели).**
При наличии таблицы loans. Выведите название книги, фамилию автора, фамилию читателя и дату выдачи. Если книга не выдана никому, читатель и дата пусты. Используйте LEFT JOIN для loans.

После успешного выполнения этих задач вы освоите главный инструмент для работы со связанными данными. В следующем занятии разберём подзапросы – запросы внутри запросов.

Занятие 7. Подзапросы (вложенные запросы)

Цель занятия: научиться использовать запросы внутри других запросов для решения задач, которые невозможно решить одним простым SELECT, а также для более гибкой фильтрации и вычислений.

7.1. Что такое подзапрос

Подзапрос (или вложенный запрос) – это запрос SELECT, который находится внутри другого SQL-запроса. Результат подзапроса используется внешним запросом.

Подзапросы могут возвращать:

- одно значение (одну строку с одним столбцом) – скалярный подзапрос;
- один столбец с несколькими строками – используется с операторами IN, ANY, ALL;
- целую таблицу – используется в FROM или JOIN.

7.2. Подзапросы в WHERE (фильтрация)

Пример 1. Найти книги, количество страниц в которых больше среднего:

```
SELECT name_book, pages
FROM books
WHERE pages > (SELECT AVG(pages) FROM books);
```

Сначала выполняется подзапрос (SELECT AVG(pages) FROM books), возвращается одно число, затем внешний запрос сравнивает pages с этим числом.

Пример 2. Найти авторов, у которых есть книги, изданные после 1870 года:

```
SELECT surname, name
FROM authors
WHERE id_author IN (SELECT DISTINCT id_author FROM books WHERE year > 1870);
```

Пример 3. Найти авторов, у которых **нет** книг (используя NOT IN):

```
SELECT surname, name
FROM authors
WHERE id_author NOT IN (SELECT DISTINCT id_author FROM books WHERE id_author IS NOT NULL);
```

7.3. Подзапросы с EXISTS и NOT EXISTS

EXISTS проверяет, возвращает ли подзапрос хотя бы одну строку. Это эффективнее IN, когда подзапрос большой, а связь простая.

Пример 4. Найти авторов, у которых есть хотя бы одна книга (более читаемая форма):

```
SELECT surname, name
```

```
FROM authors a
WHERE EXISTS (SELECT 1 FROM books b WHERE b.id_author = a.id_author);
```

Пример 5. Найти авторов, у которых нет книг (аналог NOT IN с осторожностью к NULL):

```
SELECT surname, name
FROM authors a
WHERE NOT EXISTS (SELECT 1 FROM books b WHERE b.id_author = a.id_author);
```

Преимущество EXISTS перед IN: IN срabатывает медленнее при больших наборах и требует обработки NULL. EXISTS проверяет наличие строк и часто оптимизируется лучше.

7.4. Подзапросы в SELECT (скалярные подзапросы)

Можно использовать подзапрос для вычисления значения для каждой строки.

Пример 6. Вывести книгу и количество книг того же автора (связанный подзапрос):

```
SELECT name_book, id_author,
       (SELECT COUNT(*) FROM books b2 WHERE b2.id_author = b1.id_author) AS author_book_count
FROM books b1;
```

7.5. Подзапросы в FROM (табличные подзапросы)

Подзапрос может выступать в роли временной таблицы.

Пример 7. Найти среднее количество страниц по авторам, а затем из этого результата выбрать только строки, где среднее больше 500:

```
SELECT author_id, avg_pages
FROM (SELECT id_author AS author_id, AVG(pages) AS avg_pages FROM books GROUP BY id_author) AS subq
WHERE avg_pages > 500;
```

Такие подзапросы полезны, когда нужно выполнить несколько шагов агрегации или фильтрации, которые нельзя выразить одним GROUP BY с HAVING.

7.6. Подзапросы с ANY и ALL

- > ANY(...) – больше хотя бы одного значения из подзапроса.
- > ALL(...) – больше всех значений.

Пример 8. Найти книги, у которых страниц больше, чем у любой книги автора с id_author = 1 (т.е. больше максимального количества страниц среди книг Пушкина):

```
SELECT name_book, pages
FROM books
WHERE pages > ALL (SELECT pages FROM books WHERE id_author = 1 AND pages IS NOT NULL);
```

7.7. Практический пример: несколько подзапросов

```
-- 1. Книги, год издания которых меньше среднего года всех книг
SELECT name_book, year
FROM books
WHERE year < (SELECT AVG(year) FROM books);

-- 2. Авторы, у которых есть книги с количеством страниц больше 800
SELECT surname, name
FROM authors
WHERE id_author IN (SELECT id_author FROM books WHERE pages > 800);

-- 3. Авторы, у которых максимальное количество страниц среди их книг больше 1000
SELECT a.surname, a.name
FROM authors a
WHERE (SELECT MAX(pages) FROM books b WHERE b.id_author = a.id_author) > 1000;
```

7.8. Особенности PostgreSQL

- Подзапросы могут быть некоррелированными (не зависят от внешнего запроса) и коррелированными (ссылаются на столбцы внешнего запроса, выполняются для каждой строки).
- Коррелированные подзапросы могут быть медленными на больших таблицах; иногда их можно переписать через JOIN или LATERAL.
- EXISTS обычно быстрее IN при проверке на существование.
- PostgreSQL позволяет использовать LATERAL для подзапросов, которые обращаются к столбцам предыдущих таблиц в FROM, но это тема для продвинутого уровня.

Задачи для закрепления (Занятие 7)

Решите следующие задачи в боте SQLearningBot. Они проверят умение использовать подзапросы разных типов.

1. **Скалярный подзапрос в WHERE.** Найдите книги, у которых количество страниц больше среднего. Выведите название и количество страниц.
2. **Подзапрос с IN.** Выведите фамилии и имена авторов, у которых есть книги, изданные после 1860 года.
3. **Подзапрос с NOT IN.** Выведите фамилии и имена авторов, у которых **нет** книг (т.е. они не встречаются в таблице books).
4. **Подзапрос с EXISTS.** Перепишите задачу 2, используя EXISTS вместо IN.
5. **Подзапрос с NOT EXISTS.** Перепишите задачу 3, используя NOT EXISTS.
6. **Подзапрос в SELECT (коррелированный).** Выведите название книги, год издания и количество книг того же автора (добавьте столбец same_author_count).
7. **Подзапрос в FROM.** Найдите авторов, у которых средний год издания книг позже 1850 года. Используйте подзапрос в FROM, который

- группирует books, а затем внешний SELECT фильтрует по avg_year > 1850. Выведите id_author и средний год.
8. **Подзапрос с ALL.** Найдите книги, у которых количество страниц больше, чем **у всех** книг автора с id_author = 1 (Пушкин). Выведите название, количество страниц и id_author.
 9. **Подзапрос с ANY.** Найдите книги, у которых количество страниц больше, чем **хотя бы у одной** книги автора с id_author = 2 (Толстой). Выведите название, страницы и id_author.
 10. **Вложенный подзапрос (два уровня).** Найдите авторов, у которых максимальное количество страниц среди их книг больше, чем среднее количество страниц всех книг. Подсказка: сначала найдите среднее всех книг (самый глубокий подзапрос), затем найдите максимальные страницы по авторам и сравните.
 11. **Подзапрос с агрегацией и сравнением строк.** Выведите фамилию автора и название самой толстой книги этого автора (с максимальным pages). Используйте подзапрос для нахождения максимального pages для каждого автора.
 12. **Комбинация JOIN и подзапроса.** Выведите фамилию автора, название книги и количество книг этого автора (используйте подзапрос в SELECT или JOIN с подзапросом). Результат отсортируйте по фамилии.
-

После освоения подзапросов вы сможете решать очень широкий круг задач, включая многоступенчатую фильтрацию и агрегацию. В следующем занятии разберём представления (VIEW) – сохранённые запросы, которые можно использовать как виртуальные таблицы.

Занятие 8. Представления (VIEW)

Цель занятия: научиться создавать виртуальные таблицы – представления, которые хранят готовые запросы и позволяют работать с ними как с обычными таблицами, упрощая повторное использование сложных выборок.

8.1. Что такое представление

Представление (VIEW) – это поименованный сохранённый запрос. Оно не хранит данные самостоятельно, а только «смотрит» на данные в базовых таблицах. При обращении к представлению выполняется сохранённый запрос, и вы получаете актуальные данные на текущий момент.

Представления полезны для:

- упрощения сложных запросов (особенно с JOIN, агрегатами, подзапросами);
- ограничения доступа (можно показать пользователю только определённые столбцы или строки);
- обеспечения уровня абстракции (при изменении структуры таблиц можно изменить представление, а программы, использующие его, не трогать).

8.2. Создание представления: CREATE VIEW

Синтаксис:

```
CREATE VIEW имя_представления AS  
SELECT ... ;
```

Пример 1. Создадим представление, которое показывает книги вместе с фамилиями авторов:

```
CREATE VIEW books_with_authors AS  
SELECT b.id_book, b.name_book, b.year, b.pages, a.surname, a.name  
FROM books b  
JOIN authors a ON b.id_author = a.id_author;
```

После этого к представлению можно обращаться как к обычной таблице:

```
SELECT * FROM books_with_authors;  
SELECT name_book, surname FROM books_with_authors WHERE year > 1850;
```

Пример 2. Представление с агрегацией: количество книг по авторам:

```
CREATE VIEW author_book_counts AS  
SELECT a.id_author, a.surname, a.name, COUNT(b.id_book) AS book_count  
FROM authors a  
LEFT JOIN books b ON a.id_author = b.id_author  
GROUP BY a.id_author, a.surname, a.name;
```

Использование:

```
SELECT surname, book_count FROM author_book_counts WHERE book_count > 2;
```

8.3. Изменение и удаление представления

Замена представления (если нужно изменить запрос):

```
CREATE OR REPLACE VIEW books_with_authors AS
SELECT b.id_book, b.name_book, b.year, b.pages, a.surname, a.name, a.patronymic
FROM books b
JOIN authors a ON b.id_author = a.id_author;
```

Удаление представления:

```
DROP VIEW books_with_authors;
```

8.4. Материализованные представления (Materialized Views)

В PostgreSQL есть особый вид представлений – **материализованные**. Они физически хранят данные, что ускоряет чтение, но требует обновления (REFRESH MATERIALIZED VIEW). Это полезно для медленных сложных запросов, которые не требуют абсолютно свежих данных (например, отчёты).

```
CREATE MATERIALIZED VIEW author_stats AS
SELECT a.surname, COUNT(b.id_book) AS cnt, AVG(b.pages) AS avg_pages
FROM authors a
LEFT JOIN books b ON a.id_author = b.id_author
GROUP BY a.id_author, a.surname;
```

```
-- Обновление данных в материализованном представлении
REFRESH MATERIALIZED VIEW author_stats;
```

Обычные представления (не материализованные) всегда показывают актуальные данные, но могут работать медленнее при каждом запросе.

8.5. Ограничения представлений

- Обычное представление нельзя напрямую использовать для INSERT, UPDATE, DELETE, если запрос содержит JOIN, агрегацию, DISTINCT и т.д. (хотя есть исключения с правилами или триггерами, но это сложная тема).
- Представления могут быть вложенными (представление на основе другого представления).
- При изменении базовых таблиц представление автоматически отражает изменения (кроме материализованных).

8.6. Практический пример: создание и использование представлений

```
-- 1. Представление: читатели с полным именем (Фамилия Имя Отчество)
CREATE VIEW full_readers AS
SELECT id_reader,
       CONCAT(surname, ' ', name, ' ', COALESCE(patronymic, '')) AS full_name,
```

```

birth
FROM readers;

-- Использование
SELECT full_name, birth FROM full_readers ORDER BY birth;

-- 2. Представление: статистика книг по десятилетиям
CREATE VIEW books_by_decade AS
SELECT (year / 10) * 10 AS decade,
       COUNT(*) AS books_count,
       AVG(pages) AS avg_pages
FROM books
WHERE year IS NOT NULL
GROUP BY (year / 10) * 10
ORDER BY decade;

-- 3. Представление: авторы, у которых больше 2 книг (используя предыдущее представление author_book_counts)
CREATE VIEW prolific_authors AS
SELECT surname, name, book_count
FROM author_book_counts
WHERE book_count > 2;

```

8.7. Особенности PostgreSQL

- CREATE OR REPLACE VIEW работает только если новое представление имеет те же имена и типы столбцов (или совместимые). Для существенных изменений лучше DROP VIEW и создать заново.
- У представлений можно задавать параметры WITH CHECK OPTION для обновляемых представлений – это тема для продвинутого уровня.
- Материализованные представления поддерживают индексы, что ускоряет запросы к ним.

Задачи для закрепления (Занятие 8)

Решите следующие задачи в боте SQLearningBot. Они проверят умение создавать и использовать представления.

1. **Создание простого представления.** Создайте представление active_books, которое содержит все столбцы таблицы books, но только те книги, у которых количество страниц больше 100.
2. **Использование представления.** Напишите запрос к представлению active_books, который выводит названия книг, изданных после 1850 года.
3. **Представление с JOIN.** Создайте представление book_author_info, которое выводит: название книги (name_book), год издания, фамилию автора и имя автора. Используйте JOIN.

4. **Представление с агрегацией.** Создайте представление `author_avg_pages`, которое для каждого автора выводит `id_author`, фамилию и среднее количество страниц его книг (`avg_pages`). Округлите среднее до целого.
 5. **Замена представления.** Измените представление `active_books`, добавив условие `pages IS NOT NULL` (т.е. книги без указания страниц не попадают). Используйте `CREATE OR REPLACE VIEW`.
 6. **Удаление представления.** Напишите команду для удаления представления `active_books`.
 7. **Представление на основе представления.** Сначала создайте представление `books_after_1850`, которое содержит книги, изданные после 1850 года (все столбцы). Затем создайте представление `long_books_after_1850`, которое на основе первого представления выбирает книги с количеством страниц больше 500. Выполните выборку из второго представления.
 8. **Материализованное представление (ознакомительное).** Создайте материализованное представление `mv_author_stats`, которое для каждого автора выводит фамилию, максимальное количество страниц среди его книг и минимальный год издания. Напишите команду для обновления этого представления.
 9. **Практическое использование представления для отчёта.** Создайте представление `reader_loan_stats`, которое для каждого читателя (из таблицы `readers`) выводит его фамилию и количество взятых книг (используя таблицу `loans`, если она создана). Если читатель не брал книг, количество должно быть 0. Если таблицы `loans` нет, создайте её по образцу из занятия 1.
 10. **Представление с вычисляемым столбцом.** Создайте представление `books_with_price_estimate`, которое добавляет к таблице `books` вычисляемый столбец `estimated_price` по формуле:
 - если `pages < 100`, то цена 200 рублей;
 - если `pages` от 100 до 300, то 500 рублей;
 - если `pages > 300`, то 800 рублей.Выведите все данные из этого представления.
 11. **Фильтрация через представление.** Создайте представление `authors_with_books`, которое содержит только тех авторов, у которых есть хотя бы одна книга. Используйте `EXISTS`. Затем сделайте выборку из этого представления, отсортировав авторов по фамилии.
 12. **Комплексное задание.** Создайте представление `full_library_info`, которое объединяет информацию о книгах, авторах и читателях (через `loans`). Представление должно содержать: название книги, фамилию автора, фамилию читателя, дату выдачи. Если книга не выдана, читатель и дата должны быть `NULL`. Выполните выборку из представления, ограничив её 10 записями.
-

Представления значительно упрощают работу со сложными запросами и делают код чище. В следующем занятии разберём индексы – механизмы ускорения поиска данных.

Занятие 9. Индексы и оптимизация запросов

Цель занятия: понять, что такое индексы, зачем они нужны, как их создавать и удалять, а также научиться анализировать производительность запросов с помощью EXPLAIN.

9.1. Что такое индекс

Индекс – это структура данных, которая ускоряет поиск и сортировку строк в таблице. Индекс работает по аналогии с алфавитным указателем в книге: вместо того чтобы просматривать каждую страницу, вы сразу переходите к нужному разделу.

Без индекса PostgreSQL при запросе с условием WHERE выполняет **последовательное сканирование** (Seq Scan) – читает всю таблицу целиком. С индексом – **индексное сканирование** (Index Scan) – читает только нужные строки.

Когда индексы полезны:

- фильтрация по столбцу (WHERE column = value)
- сортировка (ORDER BY column)
- соединение таблиц (JOIN по ключу)
- поиск по диапазону (BETWEEN, >, <)

Когда индексы вредны:

- таблица очень маленькая (Seq Scan быстрее)
- частые вставки/обновления/удаления (индексы нужно обновлять)
- столбец имеет мало уникальных значений (например, gender с M/F)

9.2. Типы индексов в PostgreSQL

Тип	Когда используется
B-Tree (по умолчанию)	равенство, диапазоны, сортировка
Hash	только равенство (быстрее B-Tree для =, но не поддерживает > или ORDER BY)
GiST, GIN, BRIN	специализированные (полнотекстовый поиск, массивы, JSONB, геоданные)

В начальном курсе достаточно знать про **B-Tree** – он подходит для большинства задач.

9.3. Создание и удаление индексов

Создание индекса:

```
CREATE INDEX имя_индекса ON таблица (столбец);
```

Пример 1. Создадим индекс для ускорения поиска по полю year в таблице books:

```
CREATE INDEX idx_books_year ON books(year);
```

Составной индекс по нескольким столбцам (важен порядок):

```
CREATE INDEX idx_books_author_year ON books(id_author, year);
```

Уникальный индекс (аналог UNIQUE CONSTRAINT):

```
CREATE UNIQUE INDEX idx_unique_book_name ON books(name_book);
```

Удаление индекса:

```
DROP INDEX idx_books_year;
```

9.4. Анализ запросов: EXPLAIN

Команда EXPLAIN показывает, как PostgreSQL планирует выполнять запрос. Это главный инструмент для понимания, нужен ли индекс.

Пример 2. Проанализируем запрос без индекса:

```
EXPLAIN SELECT * FROM books WHERE year = 1869;
```

Вывод может быть похож на:

```
Seq Scan on books (cost=0.00..23.40 rows=2 width=...)
Filter: (year = 1869)
```

Seq Scan – таблица сканируется последовательно, что медленно при большом количестве строк.

После создания индекса:

```
CREATE INDEX idx_books_year ON books(year);
EXPLAIN SELECT * FROM books WHERE year = 1869;
```

Вывод станет:

```
Index Scan using idx_books_year on books (cost=0.00..8.27 rows=2 width=...)
Index Cond: (year = 1869)
```

Index Scan – используется индекс, запрос быстрее.

Дополнительно: EXPLAIN ANALYZE выполняет запрос и показывает реальную статистику (стоимость, время, количество строк). Используйте осторожно на больших таблицах.

9.5. Когда индексы не используются

Даже если индекс создан, PostgreSQL может его не использовать, если:

- условие WHERE использует функцию на столбце (например, WHERE UPPER(name) = 'IVAN'). Решение – создать индекс на выражении.
- таблица очень маленькая (дешевле просканировать всю таблицу).
- тип условия не поддерживается индексом (например, LIKE '%abc' – поиск по подстроке, где % в начале, не использует B-Tree).
- столбец имеет слишком мало уникальных значений (низкая селективность).

9.6. Индексы и внешние ключи

В PostgreSQL индексы на внешние ключи **не создаются автоматически**. Если вы часто объединяете таблицы по внешнему ключу (например, books и authors по id_author), стоит создать индекс на столбце внешнего ключа:

```
CREATE INDEX idx_books_id_author ON books(id_author);
```

Это ускорит JOIN и каскадные операции (ON DELETE CASCADE).

9.7. Практический пример: создание индексов для «Library»

```
-- Индекс для поиска книг по году
CREATE INDEX idx_books_year ON books(year);

-- Составной индекс: сортировка по автору, затем по году
CREATE INDEX idx_books_author_year ON books(id_author, year);

-- Индекс на внешний ключ (ускоряет JOIN с authors)
CREATE INDEX idx_books_id_author ON books(id_author);

-- Индекс для поиска читателей по дате рождения
CREATE INDEX idx_readers_birth ON readers(birth);

-- Уникальный индекс на названия книг (если гарантируем уникальность)
CREATE UNIQUE INDEX idx_unique_book_name ON books(name_book);
```

9.8. Особенности PostgreSQL

- Индексы не обновляются мгновенно при изменении данных, но PostgreSQL делает это эффективно в рамках транзакций.
- Можно создавать **частичные индексы** (только для части строк):
CREATE INDEX idx_recent_books ON books(year) WHERE year > 2000;

- Можно создавать **индексы на выражениях**:
`CREATE INDEX idx_lower_name ON books(LOWER(name_book));` – ускоряет поиск с `LOWER()`.
 - Для просмотра всех индексов таблицы: `\d books` в `psql` или `SELECT * FROM pg_indexes WHERE tablename = 'books';`
-

Задачи для закрепления (Занятие 9)

Решите следующие задачи в боте SQLearningBot. Они проверят понимание индексов и умение анализировать запросы.

1. **Создание простого индекса.** Создайте индекс с именем `idx_books_pages` на столбце `pages` таблицы `books`.
 2. **Составной индекс.** Создайте составной индекс `idx_books_author_year_desc` на таблице `books` по столбцу `id_author` (по возрастанию) и `year` (по убыванию).
 3. **Индекс на внешний ключ.** Создайте индекс на столбце `id_author` таблицы `books`. Объясните, для какой операции этот индекс полезен.
 4. **Уникальный индекс.** Добавьте уникальный индекс на столбце `surname` таблицы `authors`. Будет ли он полезен? В каких случаях может возникнуть ошибка?
 5. **Удаление индекса.** Напишите команду для удаления индекса `idx_books_pages`.
 6. **Индекс на выражение.** Создайте индекс `idx_books_upper_name` на выражении `UPPER(name_book)`. Для какого запроса он ускорит поиск? Напишите этот запрос.
 7. **Частичный индекс.** Создайте частичный индекс `idx_recent_books` для книг, изданных после 1900 года (только для этих строк) по столбцу `year`.
 8. **Просмотр информации об индексах.** Напишите запрос к системному каталогу `pg_indexes`, который выводит все индексы таблицы `books`.
 9. **Оценка эффективности.** У вас есть таблица `books` с миллионом записей. Какой индекс вы создадите для ускорения запроса:
`SELECT * FROM books WHERE name_book = 'Война и мир';`
 10. **Практическое задание со звёздочкой.** Создайте индекс для ускорения следующего запроса:
`SELECT id_author, COUNT(*) FROM books WHERE year > 1850 GROUP BY id_author;`
(Подсказка: подумайте о составном индексе, который покрывает и фильтрацию, и группировку.)
-

Индексы – мощный инструмент, но использовать их нужно осознанно. На практике сначала пишут запрос, измеряют производительность, а затем добавляют индексы на проблемные места. В следующем занятии разберём транзакции – механизм, обеспечивающий надёжность операций с данными.

Занятие 10. Транзакции (BEGIN, COMMIT, ROLLBACK)

Цель занятия: понять, что такое транзакции, зачем они нужны, как управлять ими с помощью команд BEGIN, COMMIT и ROLLBACK, а также ознакомиться с понятиями атомарности, согласованности, изоляции и долговечности (ACID).

10.1. Что такое транзакция

Транзакция – это группа операций SQL, которая выполняется как единое целое. Либо все операции транзакции успешно применяются, либо не применяется ни одна. Транзакции необходимы для обеспечения целостности данных в ситуациях, когда несколько изменений должны быть выполнены вместе или отменены вместе.

Классический пример: перевод денег с одного счёта на другой. Операция состоит из двух шагов:

1. Вычесть сумму со счёта отправителя.
2. Добавить сумму на счёт получателя.

Если после первого шага произойдёт сбой (отключение электричества, ошибка), деньги пропадут. Транзакция гарантирует, что либо оба шага выполнятся, либо ни один.

10.2. Команды управления транзакциями (TCL)

Команда	Назначение
BEGIN (или START TRANSACTION)	начать транзакцию
COMMIT	зафиксировать все изменения, сделанные в транзакции
ROLLBACK	отменить все изменения, сделанные в транзакции с момента BEGIN
SAVEPOINT	установить промежуточную точку сохранения (можно откатиться до неё)

10.3. Простой пример транзакции

```
BEGIN;  
UPDATE accounts SET balance = balance - 100 WHERE id = 1;  
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
COMMIT;
```

Если между BEGIN и COMMIT произойдёт ошибка (например, на втором UPDATE не хватает прав), можно выполнить ROLLBACK и отменить первое изменение.

10.4. Использование ROLLBACK

```
BEGIN;  
  
DELETE FROM books WHERE year < 1800;  
  
-- Ой, передумали, давайте вернём всё обратно  
ROLLBACK;
```

После ROLLBACK таблица books останется в том же состоянии, что и до BEGIN.

10.5. Точки сохранения (SAVEPOINT)

Позволяют откатить не всю транзакцию, а только её часть.

```
BEGIN;  
  
INSERT INTO books (name_book, year) VALUES ('Тестовая книга', 2024);  
SAVEPOINT before_update;  
  
UPDATE books SET year = 2025 WHERE name_book = 'Тестовая книга';  
  
-- Обнаружили, что год не тот, откатываемся до savepoint  
ROLLBACK TO before_update;  
  
COMMIT; -- зафиксируется только вставка, обновление отменено
```

10.6. Свойства ACID

Транзакции в PostgreSQL (и других серьёзных СУБД) удовлетворяют свойствам **ACID**:

Свойство	Описание
Atomicity (Атомарность)	Транзакция выполняется целиком или не выполняется вообще
Consistency (Согласованность)	Транзакция переводит базу данных из одного согласованного состояния в другое (не нарушает ограничения, внешние ключи, проверки)
Isolation (Изоляция)	Параллельные транзакции не мешают друг другу; результат такой, как если бы они выполнялись последовательно
Durability (Долговечность)	После COMMIT изменения сохраняются даже при сбое питания

10.7. Уровни изоляции

PostgreSQL поддерживает несколько уровней изоляции (от самого слабого к самому сильному):

- READ UNCOMMITTED – в PostgreSQL фактически работает как READ COMMITTED.
- READ COMMITTED (по умолчанию) – видны только зафиксированные изменения других транзакций.
- REPEATABLE READ – в рамках транзакции все запросы видят один и тот же снимок данных на момент начала транзакции.
- SERIALIZABLE – самый строгий уровень, эмулирует последовательное выполнение транзакций (может завершаться ошибкой сериализации при конфликтах).

Уровень изоляции можно задать:

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
-- ... операции ...  
COMMIT;
```

Для большинства учебных задач достаточно уровня по умолчанию (READ COMMITTED).

10.8. Автоматические транзакции в PostgreSQL

В PostgreSQL каждая отдельная команда INSERT, UPDATE, DELETE, SELECT выполняется внутри неявной транзакции, если явно не указан BEGIN. Поэтому изменения, сделанные одной командой, фиксируются сразу.

Если нужно объединить несколько команд в одну транзакцию, обязательно используйте BEGIN / COMMIT (или START TRANSACTION).

10.9. Практический пример: транзакции в «Library»

```
-- Пример 1. Добавление нового автора и его книги в одной транзакции  
BEGIN;
```

```
INSERT INTO authors (surname, name, patronymic, birth)  
VALUES ('Чехов', 'Антон', 'Павлович', '1860-01-29');
```

```
INSERT INTO books (name_book, id_author, year, pages)  
VALUES ('Вишнёвый сад', (SELECT id_author FROM authors WHERE surname = 'Чехов'), 1903, 96);
```

```
COMMIT;
```

```
-- Пример 2. Ошибочное обновление – откат  
BEGIN;
```

```
UPDATE books SET year = year + 100 WHERE id_author = 1; -- сдвинули годы Пушкина вперёд на 100 лет
```

```

-- Проверяем: SELECT * FROM books WHERE id_author = 1;
-- Понимаем, что ошиблись

ROLLBACK; -- всё вернулось как было

-- Пример 3. Использование SAVEPOINT
BEGIN;

INSERT INTO readers (surname, name, birth) VALUES ('Смирнов', 'Алексей', '1992-03-15');
SAVEPOINT after_reader;

INSERT INTO loans (id_book, id_reader, loan_date)
VALUES (1, (SELECT id_reader FROM readers WHERE surname = 'Смирнов'), CURRENT_DATE);

-- Если у читателя уже есть невозвращённая книга (допустим, проверяем), то откатываем только
о вставку в loans
ROLLBACK TO after_reader;

COMMIT; -- читатель сохранился, выдача книги – нет

```

10.10. Особенности PostgreSQL

- PostgreSQL поддерживает вложенные транзакции через SAVEPOINT (но не через BEGIN внутри BEGIN).
- Команды DDL (CREATE TABLE, ALTER TABLE и т.д.) могут участвовать в транзакциях и откатываться (в отличие от некоторых других СУБД, где DDL не транзакционна).
- При конфликте транзакций PostgreSQL может выдавать ошибку could not serialize access на уровне SERIALIZABLE.
- Для просмотра активных транзакций: SELECT * FROM pg_stat_activity WHERE xact_start IS NOT NULL;

Задачи для закрепления (Занятие 10)

Решите следующие задачи в боте SQLLearningBot. Они проверят понимание транзакций и умение их применять.

1. **Простая транзакция с COMMIT.** Напишите транзакцию, которая добавляет нового автора «Булгаков Михаил Афанасьевич» (год рождения 1891, год смерти 1940) и его книгу «Мастер и Маргарита» (год 1966, страниц 480). Оба действия должны быть в одной транзакции. Завершите транзакцию фиксацией.
2. **Транзакция с ROLLBACK.** Напишите транзакцию, которая удаляет все книги, изданные до 1800 года, а затем откатывает это удаление. Убедитесь, что после отката книги остались на месте.
3. **Транзакция с двумя обновлениями.** Напишите транзакцию, которая:

- увеличивает количество страниц у всех книг Толстого (id_author = 2) на 50;
 - уменьшает год издания всех книг Пушкина (id_author = 1) на 1. Затем зафиксируйте изменения.
4. **Использование SAVEPOINT.** Напишите транзакцию, которая:
- вставляет нового читателя «Лебедев Сергей» (дата рождения 1985-10-10);
 - устанавливает точку сохранения;
 - пытается выдать этому читателю книгу с id_book = 999 (заведомо несуществующую, что вызовет ошибку внешнего ключа);
 - откатывается до точки сохранения;
 - фиксирует транзакцию.
- В результате читатель должен сохраниться, а ошибочная выдача – нет.
5. **Проверка атомарности.** Напишите транзакцию, которая пытается обновить pages у книги с id_book = 1 и одновременно вставить книгу с id_author = 999 (несуществующий автор). Транзакция должна завершиться ошибкой и откатиться. Зафиксируйте, что первое изменение (pages) не применилось.
6. **Вложенные транзакции через SAVEPOINT.** Напишите транзакцию, которая:
- вставляет автора «Грибоедов»;
 - устанавливает точку сохранения sp1;
 - вставляет его книгу «Горе от ума»;
 - устанавливает точку сохранения sp2;
 - обновляет год издания этой книги на 1824;
 - откатывается до sp1 (т.е. отменяет и книгу, и обновление);
 - фиксирует транзакцию.
- Какие изменения останутся в базе?
7. **DDL в транзакции.** Напишите транзакцию, которая создаёт новую таблицу temp_log с одним столбцом msg TEXT, вставляет в неё строку 'Тест', а затем откатывает транзакцию. Существует ли таблица temp_log после отката?
8. **Практическое задание на осмысление.** В библиотеке есть правило: нельзя удалить автора, если у него есть книги. Однако из-за ON DELETE CASCADE это правило нарушается. Напишите транзакцию, которая временно удаляет внешний ключ с CASCADE, удаляет автора (например, Пушкина) и все его книги, а затем восстанавливает внешний ключ (подсказка: ALTER TABLE books DROP CONSTRAINT ..., ALTER TABLE books ADD CONSTRAINT ...). В реальной практике так не делают, но для понимания транзакций полезно. **Внимание:** после восстановления внешнего ключа проверьте, что база данных осталась в согласованном состоянии.

9. **Мониторинг транзакций.** Напишите запрос к `pg_stat_activity`, который показывает все активные транзакции (где `xact_start` не `NULL`), их `pid` и время начала.
-

Транзакции – одна из важнейших концепций в работе с базами данных. Они обеспечивают надёжность и целостность даже при сбоях. В следующем, заключительном занятии, вас ждёт итоговый проект, объединяющий все изученные темы.

Занятие 11. Итоговый проект: «Библиотечная аналитика»

Цель проекта: применить все знания, полученные в занятиях 1–10, для решения комплексной задачи. Вы создадите дополнительные объекты, напишете аналитические запросы, оптимизируете их с помощью индексов и научитесь управлять транзакциями.

Оценка: 1 балл за каждую полностью верную задачу. Максимум – 13 баллов.

Проект не требует написания теории – только практика. Все решения проверяются ботом SQLearningBot.

Описание базы данных

У вас уже есть таблицы:

- authors (авторы)
- books (книги)
- readers (читатели)
- loans (выдача книг)

Начальные данные заполнены (см. Приложение А.3).

Часть 1. Подготовка данных (DDL + DML)

Задача 1.

Создайте таблицу `book_stats` для хранения статистики по книгам:

- `id_book` INT PRIMARY KEY, ссылается на `books(id_book)` с ON DELETE CASCADE
- `times_loaned` INT DEFAULT 0 (сколько раз книгу выдавали)
- `last_loan_date` DATE (дата последней выдачи, может быть NULL)

Задача 2.

Заполните таблицу `book_stats` на основе данных из `loans`:

- `times_loaned` = количество выдач этой книги
- `last_loan_date` = максимальная дата выдачи (если выдач не было – NULL)

Используйте `INSERT INTO ... SELECT ... GROUP BY`.

Задача 3.

Добавьте в таблицу `books` столбец `total_loans` типа INT DEFAULT 0.

Затем обновите этот столбец, установив значения из book_stats.times_loaned (с помощью UPDATE ... FROM или подзапроса).

Часть 2. Аналитические запросы (DQL)

Задача 4.

Выведите список книг, которые **никогда не выдавались** (total_loans = 0).
Включите: название книги, фамилию автора, год издания. Отсортируйте по году (от старых к новым).

Задача 5.

Для каждого автора выведите:

- фамилию,
- общее количество его книг,
- общее количество выдач всех его книг (сумма total_loans),
- среднее количество страниц по его книгам (округлить до целого).

Отсортируйте по убыванию общего количества выдач.
Используйте LEFT JOIN и GROUP BY.

Задача 6.

Найдите читателя (фамилию, имя), который взял больше всего книг (по количеству записей в loans). Если таких несколько – выведите любого.
Используйте подзапрос или LIMIT.

Задача 7.

Создайте представление active_loans, которое показывает:

- фамилию читателя,
 - название книги,
 - дату выдачи
- для всех **активных** выдач (return_date IS NULL).
Затем напишите запрос к этому представлению, чтобы вывести список активных выдач, отсортированный по дате выдачи (сначала самые старые).

Задача 8.

Напишите запрос, который для каждой книги выводит:

- название,
- количество дней, прошедших с последней выдачи (CURRENT_DATE - last_loan_date).

Если книга никогда не выдавалась – выведите NULL.
Используйте JOIN с book_stats.

Часть 3. Индексы и оптимизация

Задача 9.

Создайте такой индекс на столбце loan_date таблицы loans, чтобы он ускорил выполнение запроса:

```
SELECT * FROM loans WHERE loan_date BETWEEN '2025-01-01' AND '2025-12-31';
```

Задача 10.

Создайте составной индекс на таблице books по полям id_author и year.

Часть 4. Транзакции

Задача 11.

Напишите транзакцию, которая:

- добавляет нового читателя: «Смирнов Олег Иванович», дата рождения 1992-04-10;
 - выдаёт ему книгу с id_book = 3 (сегодняшняя дата);
 - фиксирует изменения.
- После выполнения проверьте (мысленно или в песочнице), что читатель появился в readers, а в loans – соответствующая запись.

Задача 12.

Напишите транзакцию с точкой сохранения:

- начать транзакцию;
 - обновить total_loans для книги с id_book = 1, установив значение 100;
 - создать точку сохранения sp1;
 - обновить total_loans для книги с id_book = 2, установив значение 200;
 - откатиться к sp1;
 - зафиксировать транзакцию.
- В результате изменения должны затронуть только книгу 1, а книга 2 остаться без изменений.
-

Часть 5. Итоговый отчёт

Задача 13.

Составьте один запрос (без CTE и оконных функций), который выводит:

- общее количество книг,
- общее количество читателей,
- количество активных выдач (где return_date IS NULL),
- общее количество выдач за всё время,
- среднее количество страниц по всем книгам (округлить до целого).

Названия столбцов: total_books, total_readers, active_loans, all_time_loans, avg_pages.

Критерии оценки

- **Отлично** – 13 баллов (все задачи верны)
 - **Хорошо** – 10–12 баллов
 - **Удовлетворительно** – 7–9 баллов
 - **Неудовлетворительно** – менее 7 баллов
-

После выполнения проекта вы закрепите все ключевые навыки работы с PostgreSQL. Поздравляем с завершением курса!

Заключение

Поздравляем! Вы прошли курс от создания таблиц до транзакций и индексов. Вы освоили синтаксис PostgreSQL, научились проектировать нормализованные структуры, уверенно выполняете DDL, DML, DQL, используете агрегацию, группировку, подзапросы, объединения, представления, понимаете транзакции и принципы ACID, умеете создавать индексы для оптимизации.

В процессе обучения вы:

- освоили синтаксис и особенности PostgreSQL;
- научились проектировать нормализованные структуры данных;
- уверенно выполняете операции DDL, DML, DQL;
- используете агрегацию, группировку, подзапросы, объединения;
- понимаете транзакции и принципы ACID;
- создаёте индексы для оптимизации запросов.

SQL – это не «мёртвый» язык, а активно развивающийся стандарт. Изученные вами приёмы применимы не только в PostgreSQL, но и в других реляционных СУБД (MySQL, Oracle, MS SQL Server) с минимальными корректировками.

Однако теория без практики быстро забывается. Регулярно решайте задачи в боте «SQLearningBot», участвуйте в соревнованиях на платформах вроде SQL-Ex, Codewars (раздел SQL) или LeetCode. Изучайте официальную документацию PostgreSQL – она считается одной из лучших среди всех СУБД и поможет углубить знания.

Желаем успехов в ваших начинаниях и надёжных баз данных!

Приложение А (справочное)

В приложении А приведены схема базы данных, SQL-скрипт для её создания и начальное наполнение данными. База данных «Library» используется во всех примерах и задачах пособия.

А.1. Схема базы данных

База данных «Library» моделирует работу библиотеки. Она состоит из следующих таблиц:

Таблица authors – хранит информацию об авторах.

Столбец	Тип	Ограничения	Описание
id_author	SERIAL	PRIMARY KEY	уникальный идентификатор автора
surname	VARCHAR(100)	NOT NULL	фамилия
name	VARCHAR(100)	NOT NULL	имя
patronymic	VARCHAR(100)		отчество (может быть NULL)
birth	DATE		дата рождения
death	DATE		дата смерти

Таблица books – хранит информацию о книгах.

Столбец	Тип	Ограничения	Описание
id_book	SERIAL	PRIMARY KEY	уникальный идентификатор книги
name_book	VARCHAR(255)	NOT NULL	название
id_author	INT	REFERENCES authors(id_author) ON DELETE CASCADE	автор (внешний ключ)
year	INT	CHECK (year > 0)	год издания
pages	INT	DEFAULT 0	количество страниц

Таблица readers – хранит информацию о читателях.

Столбец	Тип	Ограничения	Описание
id_reader	SERIAL	PRIMARY KEY	уникальный идентификатор читателя
surname	VARCHAR(100)	NOT NULL	фамилия
name	VARCHAR(100)	NOT NULL	имя
patronymic	VARCHAR(100)		отчество
birth	DATE	NOT NULL	дата рождения
registered_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	дата регистрации в библиотеке

Таблица loans – выдача книг читателям.

Столбец	Тип	Ограничения	Описание
id_loan	SERIAL	PRIMARY KEY	уникальный идентификатор выдачи
id_book	INT	REFERENCES books(id_book) ON DELETE CASCADE	книга
id_reader	INT	REFERENCES readers(id_reader) ON DELETE CASCADE	читатель
loan_date	DATE	NOT NULL	дата выдачи
return_date	DATE		дата возврата (NULL – ещё не возвращена)

A.2. Полный SQL-скрипт для создания БД

-- Удаляем таблицы, если они существуют (для чистой установки)

```
DROP TABLE IF EXISTS authors CASCADE;
DROP TABLE IF EXISTS books CASCADE;
DROP TABLE IF EXISTS readers CASCADE;
DROP TABLE IF EXISTS loans CASCADE;
```

-- Таблица авторов

```
CREATE TABLE authors (
  id_author SERIAL PRIMARY KEY,
  surname VARCHAR(100) NOT NULL,
  name VARCHAR(100) NOT NULL,
  patronymic VARCHAR(100),
```

```

    birth DATE,
    death DATE
);

-- Таблица книг
CREATE TABLE books (
    id_book SERIAL PRIMARY KEY,
    name_book VARCHAR(255) NOT NULL,
    id_author INT REFERENCES authors(id_author) ON DELETE CASCADE,
    year INT CHECK (year > 0),
    pages INT DEFAULT 0
);

-- Таблица читателей
CREATE TABLE readers (
    id_reader SERIAL PRIMARY KEY,
    surname VARCHAR(100) NOT NULL,
    name VARCHAR(100) NOT NULL,
    patronymic VARCHAR(100),
    birth DATE NOT NULL,
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица выдач книг
CREATE TABLE loans (
    id_loan SERIAL PRIMARY KEY,
    id_book INT REFERENCES books(id_book) ON DELETE CASCADE,
    id_reader INT REFERENCES readers(id_reader) ON DELETE CASCADE,
    loan_date DATE NOT NULL,
    return_date DATE
);

```

А.3. Начальное наполнение данными

```

-- Добавляем авторов
INSERT INTO authors (surname, name, patronymic, birth, death) VALUES
('Пушкин', 'Александр', 'Сергеевич', '1799-06-06', '1837-02-10'),
('Толстой', 'Лев', 'Николаевич', '1828-09-09', '1910-11-20'),
('Достоевский', 'Фёдор', 'Михайлович', '1821-11-11', '1881-02-09'),
('Гоголь', 'Николай', 'Васильевич', '1809-03-20', '1852-02-21');

-- Добавляем книги
INSERT INTO books (name_book, id_author, year, pages) VALUES
('Евгений Онегин', 1, 1833, 480),
('Капитанская дочка', 1, 1836, 320),
('Война и мир', 2, 1869, 1225),
('Анна Каренина', 2, 1877, 864),
('Преступление и наказание', 3, 1866, 672),
('Идиот', 3, 1869, 640),
('Мёртвые души', 4, 1842, 352);

-- Добавляем читателей
INSERT INTO readers (surname, name, patronymic, birth) VALUES
('Иванов', 'Иван', 'Иванович', '1990-05-15'),
('Петрова', 'Мария', 'Сергеевна', '1995-11-23'),
('Сидоров', 'Алексей', 'Петрович', '2000-01-01');

-- Добавляем несколько выдач книг
INSERT INTO loans (id_book, id_reader, loan_date, return_date) VALUES
(1, 1, '2025-01-10', '2025-01-25'),

```

```
(3, 2, '2025-02-01', '2025-02-15'),  
(5, 3, '2025-03-01', NULL),      -- ещё не вернул  
(7, 1, '2025-03-10', '2025-03-20'),  
(2, 2, '2025-03-15', NULL);
```

Приложение Б. Часто задаваемые вопросы (FAQ)

1. Нужно ли устанавливать PostgreSQL на свой компьютер?

Нет. Бот SQLearningBot проверяет запросы через AI-модель, не выполняя их в реальной СУБД. Вы можете использовать бесплатные онлайн-песочницы (например, sql-online.org, db-fiddle.com, sql-practice.com) для экспериментов, но это не обязательно.

2. Как узнать список всех таблиц в PostgreSQL?

В psql: \dt. В SQL: `SELECT tablename FROM pg_tables WHERE schemaname='public';`

3. Чем отличается SERIAL от BIGSERIAL?

SERIAL – псевдоним для INT с автоматическим инкрементом (максимум ~2 млрд). BIGSERIAL – для BIGINT (огромные значения). В учебных задачах обычно хватает SERIAL.

4. Как посмотреть план выполнения запроса?

Используйте EXPLAIN перед запросом. Для реального времени выполнения: EXPLAIN ANALYZE(выполнит запрос, используйте осторожно на больших таблицах).

5. Почему WHERE column = NULL не работает?

NULL означает «неизвестно». Для проверки на NULL используйте IS NULL или IS NOT NULL.

6. Как вставить несколько строк одним INSERT?

`INSERT INTO table (col1, col2) VALUES (1, 'a'), (2, 'b'), (3, 'c');`

7. Чем отличается DELETE от TRUNCATE?

DELETE удаляет строки по одной (можно с WHERE), можно откатить, триггеры срабатывают. TRUNCATE быстро удаляет все строки, сбрасывает последовательности, не срабатывают триггеры, но откатить можно (в транзакции).

8. Как получить первые 5 записей?

`SELECT ... LIMIT 5;` или `FETCH FIRST 5 ROWS ONLY;`

9. Что делать, если бот не засчитывает правильный запрос?

Проверьте:

- точное соответствие названий столбцов и таблиц (регистр в PostgreSQL значим, если идентификаторы не в кавычках);
- наличие ; в конце (бот может требовать или не требовать – уточните);
- пробелы и лишние символы;
- используйте подсказку в боте.

10. Можно ли использовать свой экземпляр PostgreSQL для проверки?

Да, вы можете установить PostgreSQL локально или поднять контейнер Docker. Но тогда вы будете сравнивать свои результаты с эталоном вручную.

11. Как создать индекс на несколько столбцов?

`CREATE INDEX idx_name ON table (col1, col2);` Порядок столбцов важен.

12. Где найти больше задач для тренировки?

Рекомендуемые ресурсы – в приложении В.

Приложение В. Глоссарий основных терминов

Термин	Описание
Агрегатная функция	Функция, вычисляющая одно значение по набору строк (COUNT, SUM, AVG, MIN, MAX).
Атомарность	Свойство транзакции, при котором она выполняется полностью или не выполняется вовсе.
Внешний ключ	Столбец (или набор столбцов), ссылающийся на первичный ключ другой таблицы; обеспечивает ссылочную целостность.
Индекс	Структура данных, ускоряющая поиск и сортировку строк.
Нормализация	Процесс приведения структуры БД к виду, уменьшающему дублирование и аномалии.
Первичный ключ	Столбец (или комбинация столбцов), уникально идентифицирующий каждую строку.
Представление (VIEW)	Сохранённый запрос, который можно использовать как виртуальную таблицу.
Реляционная модель	Модель данных, в которой информация хранится в таблицах, связанных ключами.
СУБД	Система управления базами данных – программное обеспечение для создания, ведения и работы с БД.

Термин	Описание
Транзакция	Группа операций SQL, выполняемая как единое целое (BEGIN, COMMIT, ROLLBACK).
JOIN	Операция объединения двух таблиц по условию (INNER, LEFT, RIGHT, FULL).
Подзапрос	Запрос SELECT внутри другого запроса.

Список использованных источников

1. Касимовская О.А. Обучение проектированию и разработке баз данных в MySQL в рамках курса по выбору для технологического профиля. – М.: МПГУ, 2021. (основа структуры и подхода)
2. PostgreSQL Global Development Group. PostgreSQL 18.3 Documentation. – URL: <https://www.postgresql.org/docs/current/> (дата обращения: 2026)
3. Жалолов О.И., Хаятов Х.У. Понятие SQL и реляционной базы данных // Universum: Технические науки. – 2020. – № 6(75). – URL: <https://7universum.com/ru/tech/archive/item/9711>
4. Пахотнова А.В. Создание и ведение БД с помощью языка SQL // Теория и практика современной науки. – 2016. – № 9(15).
5. Документация по языку SQL в PostgreSQL (перевод отдельных разделов).

Методическое пособие составлено на основе работы Касимовской О.А. (2021) и адаптировано под PostgreSQL. Актуальную версию и обновления ищите в Telegram-боте @SQLearningBot. Удачи в изучении SQL!